Manthio

Learner Frontend

Requirements Specification

Design Agency Briefing Document
Version 1.0
Released for design agency engagement

A learning platform by apigenio GmbH

Table of Contents

This document is organised into six parts, with an introductory architecture overview before Part 1. Section numbers correspond to the structure used throughout the document for cross-referencing.

System Architecture Overview

Before diving into the detailed requirements, this section provides a high-level view of how the Learner Frontend (the subject of this document) relates to the broader Manthio system. It is intended as orientation for the design agency, not as a technical specification.

Component Diagram

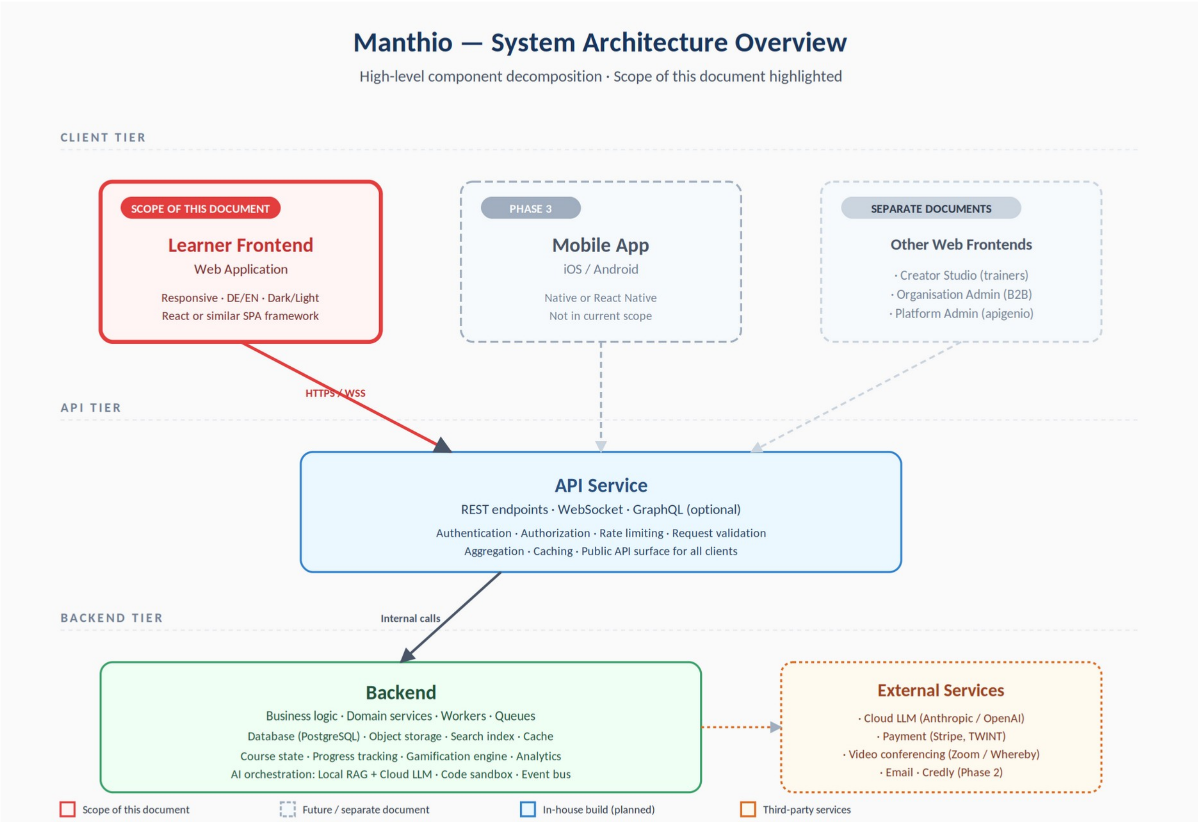


Figure 1 — Manthio system architecture. The component highlighted in red is the scope of this document.

Component Descriptions

Learner Frontend (this document)

The web application that learners use to consume courses. Built as a single-page application (React or equivalent), responsive across mobile, tablet, and desktop. Communicates exclusively with the API Service over HTTPS and WebSocket. This is the only component specified in detail in this document.

Mobile App (Phase 3)

Native iOS and Android applications planned for a later phase. Will share the same API Service as the Learner Frontend. Out of scope for the current design engagement, but the API contract is designed to accommodate it from day one.

Other Web Frontends (separate documents)

Manthio includes three additional web-based interfaces that share the same backend infrastructure but serve different user types:

- **Creator Studio:** Interface for trainers and content creators to author and publish courses. Separate requirements document.
- **Organisation Admin:** Interface for companies and schools licensing the platform, managing seats, and reviewing learner outcomes. Separate requirements document.
- **Platform Admin:** Internal interface for apigenio operators to manage the platform, monitor system health, and handle support. Separate requirements document.

API Service

The public API surface that all client applications interact with. Exposes REST endpoints for standard CRUD operations and WebSocket connections for real-time features (AI Tutor streaming, notifications, live session state). Handles authentication, authorisation, rate limiting, request validation, response aggregation, and caching. Designed and built in-house alongside the Backend.

Backend

The core application logic, data storage, and processing layer. Contains the business domain (courses, modules, progress, gamification), data persistence (PostgreSQL primary database, object storage for media, search index, cache), and background workers (analytics computation, AI orchestration, code sandbox execution, notification dispatch). The AI layer is hybrid: a self-hosted Local LLM with Retrieval-Augmented Generation (RAG) handles course-document questions cost-efficiently, while a Cloud LLM (external service) handles broader reasoning. Not directly accessible from clients — all access flows through the API Service.

External Services

Third-party services integrated for capabilities that don't make sense to build in-house:

- **Cloud LLM:** Anthropic, OpenAI, or equivalent — powers the AI Tutor's broader reasoning, code generation, and out-of-document questions. Used in conjunction with the self-hosted Local LLM described in §31.6.
- **Payment processing:** Stripe for credit cards, TWINT for Swiss payments
- **Video conferencing:** Zoom, Whereby, or equivalent — used for cohort live sessions
- **Email delivery:** Transactional email service for notifications and verification
- **Credly (Phase 2):** Digital credential issuance for course certificates

What This Document Specifies vs. Excludes

In scope (this document)	Out of scope (other documents or implementation)
Learner Frontend UX and UI	API contract and endpoint specifications
Learner Frontend behaviour and flows	Backend business logic implementation
Visual design system for the Learner Frontend	Database schema and data model
Accessibility and responsive design	DevOps, deployment, infrastructure
Component library used by Learner Frontend	Other client frontends (Creator Studio, Org Admin, Platform Admin)
Interaction patterns specific to learners	Mobile App design (Phase 3)

> *The design agency engaged via this document is responsible only for the Learner Frontend. Other components are referenced for context but will be specified, designed, and built separately.*

Part 1 — Context and Strategy

This part orients the reader. It explains what Manthio is, who it is for, and how it fits within apigenio's broader business. The design agency should read this part fully before any other; it sets the constraints that shape every later decision.

- **§1 Document Purpose and Scope:** what this document covers, who should read it, conventions used.
- **§2 Product Context:** what Manthio is, where it comes from, what differentiates it.
- **§3 Target Users and Personas:** three concrete learner personas — Sara, Marc, Tanya — and the shared needs they reveal.
- **§4 Business Model Context:** the three customer types (learner, trainer, organisation) and how this affects the Frontend.

1. Document Purpose and Scope

1.1 Purpose

This document specifies the functional and non-functional requirements for the Manthio Learner Frontend. It serves as the authoritative briefing for the design agency tasked with producing high-fidelity designs, interactive prototypes, and a final design system.

1.2 Intended Audience

- **Primary:** Design agency partners (UX designers, UI designers, art directors)
- **Secondary:** Internal stakeholders at apigenio GmbH, development team (in-house implementation)
- **Tertiary:** Future contributors, content creators, and trainers onboarding to the platform

1.3 Scope

This document covers the Learner Frontend — the interface used by individuals consuming courses on Manthio. It does not cover the following adjacent surfaces, which will be specified in separate documents:

- **Creator Studio:** Interface for trainers and content creators to author and publish courses
- **Organisation Admin:** Interface for companies/schools managing licences, users, and reporting
- **Platform Admin:** Backend administration for apigenio operators
- **Backend / API:** Data layer, authentication services, business logic — covered in technical architecture documents

1.4 Document Conventions

- **Requirement identifiers:** Each requirement is tagged with an identifier of the form REQ-AREA-NNN (e.g. REQ-DASH-001).
- **Priority:** Where priority is indicated: M = Must (MVP), S = Should, C = Could, W = Won't (this version).
- **References:** Cross-references use section numbers (e.g. "see §15.3").
- **German artefacts:** Sample copy from the student mockup is preserved in German where quoted; production copy is bilingual (DE/EN).

1.5 Document Status and Versioning

Version 1.0 — First formal release of the requirements specification, shared with the design agency to brief and scope the engagement. The document reflects extensive input: the student PRP1 final report, brand and product decisions made by apigenio, didactic research, and competitor benchmarking. Future versions will track design feedback, scope refinements, and learnings from the implementation.

> *Reading order recommendation: Design agencies should read Part 1 fully, skim Parts 2–3 for orientation, then refer to specific page sections in Part 3 and module sections in Part 4 as they design. Part 5 (user journeys) is invaluable for understanding flow continuity between screens.*

2. Product Context

2.1 What Manthio Is

Manthio is a digital learning platform that extends instructor-led courses — historically delivered as intensive in-person bootcamps — with structured self-study, AI-assisted practice, and analytics. The platform supports three delivery modes: fully self-paced, online cohorts with live video sessions, and flipped/hybrid formats that combine online self-study with focused in-person workshop time. The AI Tutor sits at the centre of the experience, augmenting (not replacing) trainer access.

2.2 Origin Story

Manthio originates from apigenio GmbH, a Swiss IT consultancy that has been delivering hands-on technical training (Python, AI fundamentals, cloud, security) in two-day intensive workshop formats. Manthio digitises these proven curricula and extends them into a scalable platform that other trainers can use to digitise their own courses.

2.3 Differentiation

Manthio's positioning rests on three differentiators:

- AI Tutor as a first-class citizen — not a sidebar widget, but a core companion that knows the current module, can grade exercises, suggest adaptive paths, and (in later phases) respond by voice.
- Multi-modal courses — the same course can be delivered as fully self-paced, as an online cohort with live video sessions, or as a flipped/hybrid format combining online self-study with in-person workshop time. The flipped format is the primary digitisation path for existing apigenio bootcamps: foundations and guided exercises move online (1–2 hour units), while in-person time concentrates on application, debugging, and project work.
- Tech-fluent UX — built for working professionals in technology, AI, and data domains. Code samples, notebooks, mathematical notation, and developer-grade tooling are first-class content types.

2.4 Etymology

The name **Manthio** derives from the Greek *manthánō* (μανθάνω) — "I learn" — which is also the root of *mathematics*. The brand carries this heritage of structured, disciplined learning into a modern, AI-augmented context.

2.5 Vision Statement

> *Manthio is the learning platform for working professionals who want to truly understand, not merely consume — combining the proven structure of intensive bootcamps with the personalisation only AI can provide.*

3. Target Users and Personas

3.1 Primary Audience

The Manthio Learner Frontend is designed primarily for working professionals in technology-adjacent fields. The platform is not optimised for casual hobby learners, children, or general consumer education at this stage.

3.2 Persona One — Sara, the Upskilling Professional

- **Age:** 32
- **Role:** Backend developer at a mid-size SaaS company
- **Context:** Her employer pays for her professional development. She picks one or two intensive courses per year.
- **Goals:** Add Python data analysis to her skill set; complete with a verifiable credential.
- **Frustrations:** Long, generic MOOC courses she never finishes; passive video walls without practice.
- **Preferred format:** Cohort-based, with peer accountability and live sessions on weekends.
- **Devices:** Laptop primary; mobile for review and notifications.

3.3 Persona Two — Marc, the Self-Paced Explorer

- **Age:** 41
- **Role:** Solutions architect, freelance
- **Context:** Pays out of pocket. Has narrow, irregular learning windows (evenings, between client work).
- **Goals:** Stay current on AI/ML developments; sample new tools and integrate them into client work.
- **Frustrations:** Cohorts that move too slowly; lack of advanced content; cannot wait for fixed start dates.
- **Preferred format:** Self-paced, on-demand, with AI Tutor for deep questions.
- **Devices:** Laptop and tablet; uses dark mode exclusively.

3.4 Persona Three — Tanya, the Bootcamp Participant

- **Age:** 28
- **Role:** Career changer, transitioning from finance into data science
- **Context:** Enrolled in a paid flipped Python bootcamp by her employer: 4 self-study units (1–2h each) plus 2 half-day in-person sessions at apigenio's training location. First week.
- **Goals:** Survive the bootcamp, score high, demonstrate competence quickly.
- **Frustrations:** Feels behind; needs help between in-person sessions; embarrassed to ask basic questions in group chat.
- **Preferred format:** Highly structured flipped format — clear rhythm between self-study and in-person practice — with private AI Tutor for judgement-free questions and direct access to her trainer for blockers.
- **Devices:** Laptop only; uses light mode.

3.5 Common Needs Across Personas

- Clarity about where they are in the course at any moment
- Quick, judgement-free access to help (AI Tutor)
- Sense of progress and momentum (gamification, streaks)
- Practical exercises, not just passive content
- Mobile access for review on the move
- Privacy — their learning struggles are not advertised to colleagues

3.6 Anti-Personas (Out of Scope)

Manthio is explicitly not designed for:

- Children under 16
- Casual hobby learners seeking entertainment
- K-12 schools or formal academic accreditation tracks
- Users with no prior digital literacy

4. Business Model Context

4.1 Three Customer Types

Manthio serves three distinct customer types, each with their own interface needs:

Customer Type	Primary Activity	Interface
Learner (individual)	Consumes courses, builds skills, earns credentials	Learner Frontend (this document)
Trainer / Course Creator	Authors and publishes courses on the platform	Creator Studio (separate spec)
Organisation (B2B)	Licences seats, assigns courses, reviews outcomes	Org Admin (separate spec)

4.2 Document Focus

This document covers exclusively the Learner Frontend. References to the other surfaces appear only where they affect the learner experience (for example, when a trainer publishes course content that the learner consumes).

4.3 Revenue Streams Affecting Learner UX

- **Direct purchase:** Learner pays for a course (one-time fee or subscription)
- **Organisation licensing:** Learner accesses courses via employer/school licence — no payment UI shown
- **Cohort enrolment:** Learner enrols in a scheduled cohort with fixed start date
- **Subscription tier:** Premium subscription unlocks all self-paced courses, AI Tutor unlimited use

The Learner Frontend must gracefully handle all four entry paths.

Part 2 — Cross-Cutting Requirements

Sections here define rules that apply across all pages and modules. The design agency should treat these as constraints when designing any single screen — colour, type, accessibility, responsive breakpoints, and the gamification layer all live here, as does the didactic framework (Bloom) that shapes content placement decisions.

- **§5 Design Principles:** ten foundational principles, including the Bloom-based didactic framework (§5.9) and the no-advertising commitment (§5.10).
- **§6 Brand and Visual Identity:** colour palettes (dark + sand-toned light), typography, voice. Refinable by the agency.
- **§7 Non-Functional Requirements:** responsive, performance, accessibility (WCAG 2.1 AA), i18n, theming, privacy.
- **§8 Global UI Components:** sidebar, top bar, footer, modals, toasts — the structural shell.
- **§9 Gamification Layer:** XP, levels, streaks, celebrations — applied with restraint, not as a primary attention-grabber.

5. Design Principles

These principles guide every design decision. When in doubt, they take precedence over visual fashion or feature ambition.

5.1 Clarity Over Decoration

Every screen must answer three questions at a glance: Where am I? What can I do? What happens next? Decorative elements that obscure these answers must be removed.

5.2 Friendly-Smart Tone

The interface speaks like a knowledgeable colleague — confident but not arrogant, warm but not childish. Reference brands for tone: Notion, Loom, Linear. Anti-references: Comic Sans-era LMS systems, gamification-heavy children's apps.

5.3 Tech-Fluent by Default

Code samples, terminal output, mathematical notation, and Markdown are first-class content. Monospaced fonts must be used wherever code appears. Syntax highlighting is non-optional.

5.4 Restrained Gamification

Gamification is present and visible (the user explicitly wants it), but it should never compete with the actual learning content for attention. XP toasts auto-dismiss; confetti is brief; level-ups celebrate without interrupting flow for more than five seconds.

5.5 Dual-Theme Native

Dark mode is the default and should feel like the primary experience. Light mode must be a true equal, not an afterthought. No element should rely on theme-specific assets that don't have a counterpart.

5.6 Accessible by Construction

WCAG 2.1 AA is the minimum bar. Colour-only signalling is forbidden. All interactive elements must be keyboard-navigable. Focus states must be visible in both themes.

5.7 Mobile is Not an Afterthought

Although Manthio is primarily used on laptops, mobile must be a genuine experience for review, notifications, and short sessions — not a degraded preview.

5.8 Speed is a Feature

Perceived performance matters. Skeleton states must appear within 100ms. Page transitions should be instant. Loading spinners are a last resort; prefer optimistic UI and skeleton screens.

5.9 Didactic Framework — Bloom's Taxonomy as the Backbone

The platform's didactic logic follows Bloom's revised taxonomy. This is not decoration — it shapes which content types belong where, how the AI Tutor responds, and why the flipped format works. The design must reflect this layering visually and behaviourally.

Cognitive Level	Best supported by	Where in Manthio
Remember	Microlearning, flashcards, spaced repetition	Self-study modules, daily review quizzes, AI Tutor recall prompts
Understand	Explanations, worked examples, concept maps	Video lessons, AI Tutor explanations, H5P interactive content
Apply	Guided exercises, code sandboxes, instant feedback	Code Sandbox (§25), inline quizzes, AI Tutor hint-driven feedback
Analyse	Comparison tasks, debugging, peer review	In-person workshop sessions, community forum, AI Tutor analysis prompts
Evaluate	Code review, critique, justification	In-person sessions, peer review (Phase 2), trainer office hours
Create	Open projects, original work	In-person sessions, project assignments, trainer-supervised work

The platform handles "Remember" and "Understand" excellently and is well-suited for "Apply". Higher levels — Analyse, Evaluate, Create — benefit from human interaction and are the explicit justification for keeping in-person time in the flipped format. The AI Tutor's behaviour adapts to the cognitive level being addressed: recall-style prompts for Remember/Understand, Socratic questioning for Apply/Analyse, and deferral to the trainer for Evaluate/Create.

> *Design implication: lesson type indicators (§14, §15) should subtly signal cognitive level. AI Tutor (§16) responses must match the level — never give the answer at Apply level when the learner should reason it out.*

5.10 No Advertising, Ever

A direct finding from learner research (see §38): advertising inside learning interfaces is among the strongest demotivators. Manthio must be free of third-party advertising, sponsored content posing as content, or paid-for course recommendations. The only commercial messages allowed are those about Manthio's own subscription plans, and even those are shown sparingly and never inside the Content Player.

6. Brand and Visual Identity

> *This section summarises brand decisions. A separate Brand Foundation Document is available with the full rationale and extended guidelines.*

6.1 Brand Name and Mark

- **Product name:** Manthio
- **Parent:** "A learning platform by apigenio GmbH" — visible in footer and at minor touchpoints
- **Logo direction:** Stylised M or the Greek letter µ (mu). To be finalised by the design agency.

6.2 Colour Palette

The colour system carries the brand into every screen. The student mockup established a strong dark mode that should be preserved as the foundation. The light mode shown below is a starting point in a warmer, sand-toned direction — a contrast to the cool light mode in the original mockup. The design agency is expected to refine both palettes.

> *These palettes are starting points, not final colour decisions. The design agency may propose alternative tones, additional shades, or refined contrast ratios. What must be preserved is the structure: named tokens with consistent semantic usage across both modes.*

6.2.1 Dark Mode (default theme — carried forward from student mockup)

The dark mode uses a near-black foundation with a vivid cyan brand accent. It feels modern, tech-fluent, and appropriate for a developer audience. This palette is the primary visual identity of Manthio.

Swatch	Token	Hex	Usage
	`--bg`	`#060809`	App background, deepest level
	`--panel`	`#0F1417`	Card and panel background
	`--panel2`	`#161B1F`	Elevated or alternate panel
	`--line`	`#1E2A32`	Borders, dividers, hairlines
	`--muted`	`#7A8FA0`	Secondary text, hints, labels
	`--text`	`#EDF2F6`	Primary text
	`--cyan`	`#00F5E4`	Brand accent, primary action, progress
	`--cyan2`	`#00C4B7`	Hover state for brand accent
	`--purple`	`#9D6CFF`	Trainer / elevated content marker
	`--yellow`	`#FFCF3F`	Streak, warning, attention
	`--red`	`#FF5561`	Error, danger, destructive action
	`--green`	`#2BDE7E`	Success, completed, positive
	`--orange`	`#FF8C42`	Featured tag, bestseller, highlight

6.2.2 Light Mode (proposed direction — sand-toned, illustrative)

The student mockup includes a functional light mode, but the result feels cool and clinical. A warmer, sand-toned light mode would better complement the dark mode's character and feel premium rather than utilitarian. The palette below is illustrative — a starting point. Token names and their semantic usage must align with the dark mode.

Swatch	Token	Hex	Usage
	`--bg`	`#FAF6EE`	App background, warm cream
	`--panel`	`#FFFFFF`	Card and panel background, pure white
	`--panel2`	`#F0EAD9`	Elevated or alternate panel, warm beige
	`--line`	`#D8CFB8`	Borders, dividers, warm gray-beige
	`--muted`	`#6E6650`	Secondary text, hints, warm muted
	`--text`	`#2A221A`	Primary text, warm near-black
	`--cyan`	`#0D7D6E`	Brand accent (deeper teal for light contrast)
	`--cyan2`	`#0A5E54`	Hover state for brand accent
	`--purple`	`#6830C0`	Trainer / elevated content marker
	`--yellow`	`#B88800`	Streak, warning, attention (gold)
	`--red`	`#B91C1C`	Error, danger, destructive action
	`--green`	`#15803D`	Success, completed, positive
	`--orange`	`#C2410C`	Featured tag, bestseller (warm rust)

6.2.3 Universal Constraints

Regardless of how the agency refines these palettes, the following constraints apply:

- Both light and dark modes must be specified together with matching token names
- Brand accent must be distinct and recognisable in both modes
- Functional colours (success, warning, error, info) must be distinguishable by users with colour-vision differences
- Contrast ratios must meet WCAG AA: 4.5:1 for normal text, 3:1 for large text and UI components
- No colour should rely solely on a theme to convey meaning (always pair with icon or text)
- The brand accent (--cyan) must remain visually distinct from the success colour (--green) in both modes

6.3 Typography

- **Display / headlines:** Currently Syne (student choice); alternatives welcome (Söhne, Geist, GT America)

- **Body / UI:** Currently DM Sans; alternatives welcome (Inter, Söhne)
- **Monospace:** JetBrains Mono is required and non-negotiable for code
- **Font size baseline:** 13px body in the student mockup is too small for professional use; baseline should be 14–15px

6.4 Voice and Tone

- **Address:** German "du", English "you" — informal
- **Length:** Short, scannable; long-form reserved for content, not UI
- **Humour:** Light, occasional, never at the user's expense
- **Emoji:** Sparing use for signals (🔥 streak, ✓ completion); avoid decoration

7. Non-Functional Requirements

7.1 Responsive Design

- **`REQ-NFR-001`** The interface must adapt to three primary breakpoints: mobile (< 768px), tablet (768–1024px), desktop (> 1024px). [M]
- **`REQ-NFR-002`** All primary flows must be completable on mobile, including course consumption, AI Tutor interaction, and quiz answering. [M]
- **`REQ-NFR-003`** Sidebar collapses to a hamburger menu below 1024px width. [M]
- **`REQ-NFR-004`** The content player adapts video, code, and PDF layouts for mobile portrait orientation. [M]

7.2 Performance

- **`REQ-NFR-010`** Largest Contentful Paint (LCP) must be under 2.5 seconds on a 4G connection. [M]
- **`REQ-NFR-011`** First Input Delay (FID) must be under 100ms. [M]
- **`REQ-NFR-012`** Cumulative Layout Shift (CLS) must be below 0.1. [M]
- **`REQ-NFR-013`** Page-to-page navigation within authenticated areas must feel instant (under 200ms perceived latency, achieved via SPA navigation and prefetching). [M]
- **`REQ-NFR-014`** Skeleton screens must appear within 100ms for any view that requires data fetching. [M]

7.3 Accessibility

- **`REQ-NFR-020`** All interfaces must meet WCAG 2.1 Level AA. [M]
- **`REQ-NFR-021`** All interactive elements must be reachable and operable by keyboard alone. [M]
- **`REQ-NFR-022`** Focus indicators must be clearly visible in both light and dark themes. [M]
- **`REQ-NFR-023`** Colour contrast ratios must meet WCAG AA (4.5:1 for normal text, 3:1 for large text). [M]
- **`REQ-NFR-024`** All content imagery and icons used for meaning must have text alternatives. [M]
- **`REQ-NFR-025`** ARIA landmarks (main, navigation, complementary, contentinfo) must be present on all pages. [M]
- **`REQ-NFR-026`** Form fields must have associated labels; error states must be announced to screen readers. [M]
- **`REQ-NFR-027`** Video content must support closed captions; the player must expose caption controls. [M]

7.4 Browser Support

- **`REQ-NFR-030`** The platform must support the latest two major versions of Chrome, Firefox, Safari, and Edge. [M]
- **`REQ-NFR-031`** iOS Safari and Android Chrome must be supported (latest two major versions). [M]

- **`REQ-NFR-032`** Internet Explorer is not supported. Legacy Edge (pre-Chromium) is not supported. [M]

7.5 Internationalisation

- **`REQ-NFR-040`** All interface strings must be externalised and translatable. [M]
- **`REQ-NFR-041`** Initial languages: German (de-CH) and English (en). [M]
- **`REQ-NFR-042`** Language is independent of region; users can choose interface language regardless of geographic location. [M]
- **`REQ-NFR-043`** Date, time, currency, and number formatting follow the user's locale (Swiss German default uses CHF, period as decimal separator, etc.). [M]
- **`REQ-NFR-044`** The design must accommodate copy length variation of up to +30% for German translations of English source. [S]

7.6 Theming

- **`REQ-NFR-050`** Dark mode is the default theme on first visit (unless the OS preference is light). [M]
- **`REQ-NFR-051`** Users can override theme preference; the choice persists across sessions. [M]
- **`REQ-NFR-052`** Theme switching must not require a page reload. [M]
- **`REQ-NFR-053`** The theme toggle is reachable from both the top bar and Settings. [M]

7.7 Data Privacy and Security

- **`REQ-NFR-060`** The platform must comply with EU GDPR and Swiss DSG. [M]
- **`REQ-NFR-061`** All learner activity data must be visible to the learner in a personal data export. [M]
- **`REQ-NFR-062`** Account deletion must be self-service and irreversible after a 30-day grace period. [M]
- **`REQ-NFR-063`** Sensitive actions (password change, payment, account deletion) require recent authentication. [M]

7.8 Offline and Resilience

- **`REQ-NFR-070`** The application must display a clear offline indicator and queue read actions for retry when connection returns. [S]
- **`REQ-NFR-071`** Quiz answers and notes typed offline must be saved locally and synced when reconnected. [S]
- **`REQ-NFR-072`** True offline mode (downloading course content) is out of scope for this version. [W]

8. Global UI Components

These components appear on most or all authenticated pages and form the structural shell of the application.

8.1 Sidebar (Primary Navigation)

8.1.1 Purpose

The sidebar is the primary navigation surface. It is visible on all authenticated pages on desktop and tablet; on mobile it slides in from the left when triggered by the hamburger menu.

8.1.2 Contents

- **Brand block:** Manthio logo + wordmark + tagline ("Premium E-Learning" or refined version)
- **XP and Level indicator:** Current level, progress bar to next level, XP count
- **Main navigation:** Dashboard, My Courses, Learning Path, AI Tutor, Analytics, Resources, Community, Settings
- **Primary CTA:** "Continue learning" button — deep-links to the user's active module
- **User mini-card:** Avatar, name, level/role label

8.1.3 States

- `REQ-SIDEBAR-001` Active page is visually distinguished from inactive items. [M]
- `REQ-SIDEBAR-002` Hover state on inactive items previews navigation. [M]
- `REQ-SIDEBAR-003` On screens narrower than 1024px, the sidebar is hidden by default and slides in via overlay. [M]
- `REQ-SIDEBAR-004` The sidebar can be collapsed to icon-only mode on desktop to reclaim horizontal space (toggle in lower section). [S]

8.2 Top Bar

8.2.1 Contents

- **Global search:** Searches courses, modules, resources, and forum threads. See §29.
- **Streak pill:** "🔥 N days" — clickable, opens streak detail
- **Theme toggle:** Switches dark/light
- **Notifications bell:** Badge with unread count; opens notification dropdown
- **User avatar:** Click opens user menu (Profile, Settings, Sign out)

8.2.2 Requirements

- `REQ-TOPBAR-001` Search input has a keyboard shortcut (Cmd/Ctrl+K) that focuses it. [M]
- `REQ-TOPBAR-002` The notification bell shows an unread count badge that updates in real time. [M]
- `REQ-TOPBAR-003` Streak pill animates on hover (subtle flame motion) to draw light attention. [S]
- `REQ-TOPBAR-004` On mobile, search collapses to an icon that expands when tapped. [M]

- `REQ-TOPBAR-005` Removed from student mockup: role switcher dropdown — role is determined by backend permissions, not user choice. [M]

8.3 Footer

8.3.1 Contents

- Brand line: "Manthio · Powered by apigenio GmbH"
- Legal links: Privacy Policy, Terms of Service, Cookie Settings, Impressum (Swiss legal requirement)
- Support link: Contact / Help Center
- Copyright year (auto-updated)

8.3.2 Requirements

- `REQ-FOOTER-001` Footer is present on all authenticated and public pages. [M]
- `REQ-FOOTER-002` Footer is visually distinct from content (background shade or top border). [M]
- `REQ-FOOTER-003` Footer collapses to a tighter mobile layout (stacked links) below 768px. [M]

8.4 Modal Overlay System

8.4.1 Modal Types Required

- **Confirmation modal:** Destructive action confirmation, course completion confirmation
- **Form modal:** Quick edit, simple data entry (e.g. add note)
- **Detail modal:** Student detail (for trainers), weakness analysis breakdown
- **Celebration modal:** Level up, course completion, milestone
- **Quiz modal:** Quiz question display with multi-step flow

8.4.2 Requirements

- `REQ-MODAL-001` All modals dim the underlying interface with a semi-transparent overlay. [M]
- `REQ-MODAL-002` Modals close on Escape key, on overlay click, and on explicit close button (X). Exception: data-entry modals require explicit dismissal to avoid accidental data loss. [M]
- `REQ-MODAL-003` Only one modal is visible at a time; opening a modal closes any other. [M]
- `REQ-MODAL-004` Modals trap keyboard focus and restore focus to the triggering element on close. [M]
- `REQ-MODAL-005` Modals adapt to mobile (full-screen sheet or bottom-sheet style). [M]

8.5 Toast Notifications

8.5.1 Toast Types

- **XP gain:** "+25 XP — Quiz answered correctly" (cyan/brand accent)
- **Upload success:** "✓ File uploaded" (green)
- **Error:** "Something went wrong, please try again" (red)

- **Info:** Generic notice (neutral)

8.5.2 Requirements

- `REQ-TOAST-001` Toasts appear in a consistent screen corner (bottom-right on desktop, top-centre on mobile). [M]
- `REQ-TOAST-002` Toasts auto-dismiss after 3 seconds for positive/info toasts, 5 seconds for error toasts. [M]
- `REQ-TOAST-003` Toasts can be dismissed manually by clicking. [M]
- `REQ-TOAST-004` Multiple toasts stack vertically; older toasts move to make room. Maximum three visible at once. [M]
- `REQ-TOAST-005` Error toasts can include an action button ("Retry"). [S]

8.6 Loading and Empty States

- `REQ-LOAD-001` Every list view has a defined empty state with helpful copy and a primary action (e.g. "You haven't started any courses yet — browse the catalog"). [M]
- `REQ-LOAD-002` Skeleton loaders mimic the structure of the final content. [M]
- `REQ-LOAD-003` Long operations (over 3 seconds) show progress indicators with cancel option where possible. [M]
- `REQ-LOAD-004` Failed loads show error states with clear retry actions. [M]

8.7 Hamburger Menu (Mobile)

- `REQ-HAM-001` Hamburger icon is positioned in the top-left corner. [M]
- `REQ-HAM-002` Tapping the hamburger slides the sidebar in from the left with a dim overlay. [M]
- `REQ-HAM-003` Sidebar can be dismissed by tapping the overlay, the hamburger again, or swiping left. [M]
- `REQ-HAM-004` When sidebar is open, the main content is not interactive. [M]

9. Gamification Layer

Gamification is a core feature of Manthio and a stated business priority. This section defines the systems that span all pages.

9.1 XP (Experience Points)

9.1.1 What Awards XP

Action	XP Award	Frequency Cap
Answer quiz question correctly	+25 XP	Per question
Complete a learning unit	+50 XP	Per unit
Complete a module	+150 XP	Per module
Complete a course	Course-specific (300–600 XP)	Per course
Maintain daily streak	+10 XP / day	Once per day
Ask AI Tutor a question	+10 XP	Max 5 per day
Submit assignment	+75 XP	Per assignment
Help peer in forum (answer marked best)	+50 XP	Unlimited

9.1.2 XP Display Requirements

- `REQ-XP-001` Current XP and progress to next level are visible at all times in the sidebar. [M]
- `REQ-XP-002` XP gains trigger a transient toast notification. [M]
- `REQ-XP-003` XP gains visibly animate the sidebar progress bar. [M]
- `REQ-XP-004` XP history is visible in Analytics (last 30 days, with source breakdown). [S]

9.2 Levels

- `REQ-LEVEL-001` Levels start at 1 and increase indefinitely. [M]
- `REQ-LEVEL-002` XP required per level follows a curve: Level N requires 10,000 × (1 + 0.1 × N) XP. (Subject to tuning.) [M]
- `REQ-LEVEL-003` Reaching a new level triggers a full-screen celebration modal with confetti animation. [M]
- `REQ-LEVEL-004` Level milestones (10, 25, 50, 100) unlock badge rewards (visual only, no functional gating). [S]
- `REQ-LEVEL-005` Level number is displayed alongside the user's name where space permits. [M]

9.3 Streaks

- **`REQ-STREAK-001`** A streak day is counted when the user completes at least one learning activity (quiz, unit, module, AI Tutor session). [M]
- **`REQ-STREAK-002`** Streaks are based on the user's local time zone, with a single "day" defined as 00:00 to 23:59 local. [M]
- **`REQ-STREAK-003`** Missing a day breaks the streak. [M]
- **`REQ-STREAK-004`** Users get one "streak freeze" per month (a missed day that doesn't break the streak). [S]
- **`REQ-STREAK-005`** Streak indicator in the top bar shows current count with flame icon. [M]
- **`REQ-STREAK-006`** Reaching streak milestones (7, 14, 30, 60, 100 days) triggers celebrations. [S]

9.4 Celebrations and Animations

- **`REQ-CELEB-001`** Confetti animation plays on: module completion, course completion, level up. [M]
- **`REQ-CELEB-002`** Confetti duration is bounded (1.5 to 2.5 seconds); user can dismiss any modal even if confetti is still playing. [M]
- **`REQ-CELEB-003`** Users with prefers-reduced-motion enabled see a static celebratory graphic instead of animation. [M]
- **`REQ-CELEB-004`** Celebration sounds are off by default and can be enabled in Settings. [S]

9.5 Badges (Phase 2)

Badges are not part of the initial release but the visual system should be designed to accommodate them in profile and Analytics.

- Achievement badges (e.g. first course, ten quizzes, top of cohort)
- Skill badges (e.g. "Python Fundamentals certified")
- Streak badges (7, 30, 100, 365 days)

9.6 Leaderboards (Phase 3)

Leaderboards are not part of the initial release but the design should leave room for them in Analytics. Privacy considerations: opt-in only, pseudonymous identifiers in public leaderboards.

Part 3 — Pages

Each section here defines one major page or surface of the Learner Frontend, with its purpose, layout, functional requirements, edge cases, and mobile behaviour. Where a screenshot from the student mockup exists, it is embedded as a visual anchor. Sections marked as "not present in mockup" require design from scratch.

- **§10 Onboarding and Authentication:** sign-up, sign-in, first-time onboarding — designed from scratch.
- **§11 Dashboard:** the daily landing surface; greeting, active course, stats, recommendations.
- **§12 Course Catalog:** discover and resume courses.
- **§13 Course Detail and Booking:** convince, book, handle three delivery modes (self-paced, online cohort, flipped), bundled subscriptions, minimum cohort size, pre-session prep.
- **§14 Learning Path:** the structured spine of a course (module cards by type, lesson-level building blocks (Bausteine), and multi-course Career Tracks for cross-course journeys).
- **§15 Content Player:** the heart of the platform — multi-format content delivery with embedded AI Tutor, Microlearning mode, and auto-completion.
- **§16 AI Tutor:** the conversational assistant with source indicator, mode toggle, and cognitive outsourcing safeguards.
- **§17 Live Session View:** synchronous events (online or in-person) plus asynchronous trainer availability.
- **§18 My Progress and Analytics:** learner-facing data — activity, weaknesses (with deep-dive integration), skill profile.
- **§19 Resources and Knowledge Base:** file access organised by course.
- **§20 Community and Forum:** Q&A per course, AI-augmented.
- **§21 Profile and Settings:** identity, account, subscription, preferences, privacy.

10. Onboarding and Authentication

> *This area was not present in the student mockup. It is critical for any production launch and must be specified by the design agency.*

10.1 Sign Up

10.1.1 Entry Points

- Marketing site "Sign up" button
- Course detail page "Enrol" button when not logged in
- Invitation link from organisation

10.1.2 Fields and Flow

- **`REQ-AUTH-001`** Email address (required, validated, unique). [M]
- **`REQ-AUTH-002`** Password (minimum 12 characters, with strength meter). [M]
- **`REQ-AUTH-003`** Full name (required, used for personalisation and certificates). [M]
- **`REQ-AUTH-004`** Acceptance of Terms of Service and Privacy Policy (required, with links). [M]
- **`REQ-AUTH-005`** Optional marketing consent (off by default). [M]
- **`REQ-AUTH-006`** Email verification flow with code or link (verified before paid actions). [M]

10.1.3 Social Sign-On

- **`REQ-AUTH-010`** Google sign-on. [M]
- **`REQ-AUTH-011`** Microsoft sign-on (relevant for corporate users). [M]
- **`REQ-AUTH-012`** GitHub sign-on (relevant for developer audience). [S]
- **`REQ-AUTH-013`** Apple sign-on. [S]
- **`REQ-AUTH-014`** SSO via SAML/OIDC for organisation accounts. [S]

10.2 Sign In

- **`REQ-AUTH-020`** Email + password sign in. [M]
- **`REQ-AUTH-021`** "Remember me" option extends session length. [M]
- **`REQ-AUTH-022`** Password reset via email link. [M]
- **`REQ-AUTH-023`** Failed sign-in attempts are rate-limited; after 5 failures, account is temporarily locked with email notification. [M]
- **`REQ-AUTH-024`** Two-factor authentication (TOTP) available in Settings. [S]

10.3 First-Time Onboarding

After successful sign-up and email verification, new users are guided through a brief onboarding sequence:

10.3.1 Step 1 — Welcome

- Greeting with first name

- 60-second video or static slide introducing Manthio's philosophy
- "Skip" available at all times

10.3.2 Step 2 — Goals

- Question: "What brings you here?" with options: Career growth, Specific skill, Curiosity, Employer assigned
- Question: "Time per week you can invest?" with options: < 2h, 2–5h, 5–10h, > 10h
- This data influences AI Tutor recommendations and dashboard messaging

10.3.3 Step 3 — Profile Picture (optional)

- Upload or initials avatar

10.3.4 Step 4 — First Recommendation

- Suggest 3 starter courses based on Step 2 answers
- "Browse all courses" alternative

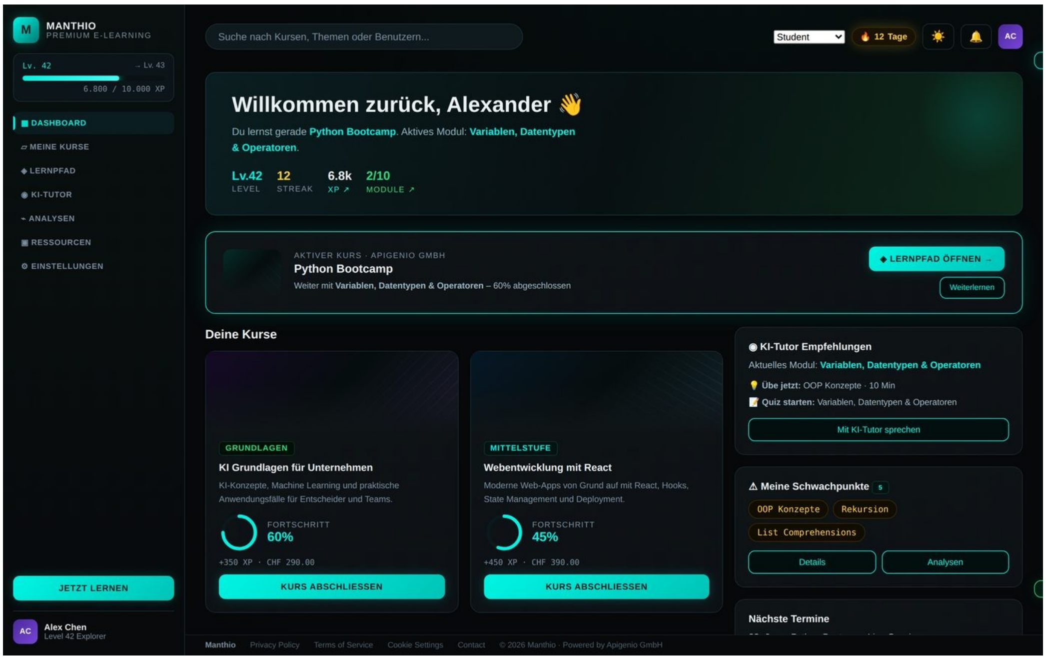
10.3.5 Requirements

- **`REQ-ONBOARD-001`** Onboarding can be skipped entirely; user is then dropped on the dashboard with prominent "complete your profile" prompts. [M]
- **`REQ-ONBOARD-002`** Each onboarding step is independently skippable. [M]
- **`REQ-ONBOARD-003`** Onboarding progress is saved; if the user closes the browser mid-flow, they resume where they left off. [M]
- **`REQ-ONBOARD-004`** Onboarding is repeatable from Settings if the user wants to revisit. [S]

10.4 Account Recovery

- **`REQ-RECOVER-001`** Password reset via email link, valid for 1 hour. [M]
- **`REQ-RECOVER-002`** Backup codes for 2FA users. [S]
- **`REQ-RECOVER-003`** Support email/chat for account recovery if all methods fail. [M]

11. Dashboard



Student mockup reference — Dashboard view (Alex Chen, Level 42, Python Bootcamp)

The Dashboard is the default landing page after sign-in. It orients the user, surfaces what's next, and provides quick access to active learning.

11.1 Purpose

In one screen, the user should know: where they are in their journey, what to do next, and what's coming up.

11.2 Layout Structure

- **Hero section:** Personal greeting, current active course/module, stats grid
- **Active course CTA card:** Large, visually distinct, deep-linked to the active module's continue point
- **Two-column layout (desktop):** Left = course cards + activity chart; Right = AI Tutor recommendations + weaknesses + upcoming events
- **Mobile:** Stacked, vertical

11.3 Functional Requirements

- `REQ-DASH-001` Greeting includes the user's first name and reflects time of day (good morning/afternoon/evening). [M]
- `REQ-DASH-002` If the user has an active course, the hero highlights the active module and continue action. [M]
- `REQ-DASH-003` If the user has no active course, the hero promotes course discovery with personalised recommendations. [M]
- `REQ-DASH-004` Stats grid shows: current level, streak, total XP, modules completed (or analogous indicators if user has multiple active courses). [M]

- `REQ-DASH-005` Stats grid items are clickable and link to the relevant section (Analytics for XP, Learning Path for modules). [M]
- `REQ-DASH-006` Course card preview shows up to 4 enrolled courses (most recently active first); a "See all" link goes to Course Catalog. [M]
- `REQ-DASH-007` Activity chart shows the last 7 days of learning time. [M]
- `REQ-DASH-008` AI Tutor recommendations card shows 2–3 contextual prompts based on current module and weaknesses, with "Open AI Tutor" CTA. [M]
- `REQ-DASH-009` Weaknesses card shows up to 3 identified skill gaps, with "Practice with Tutor" and "View details" actions. [M]
- `REQ-DASH-010` Upcoming events card shows the next 2–3 events (live sessions, assignment deadlines, cohort milestones). [M]
- `REQ-DASH-011` If the user has no upcoming events, the card is hidden rather than empty. [M]

11.4 Edge Cases

- **Brand-new user (no courses):** Hero changes to onboarding-style "Start your first course" with recommendations.
- **Returning after long break (>30 days):** Hero acknowledges absence and offers refresher options.
- **Cohort user pre-start:** Hero shows countdown to cohort start date and pre-reading materials.
- **All courses completed:** Hero celebrates and suggests next courses.

11.5 Mobile Considerations

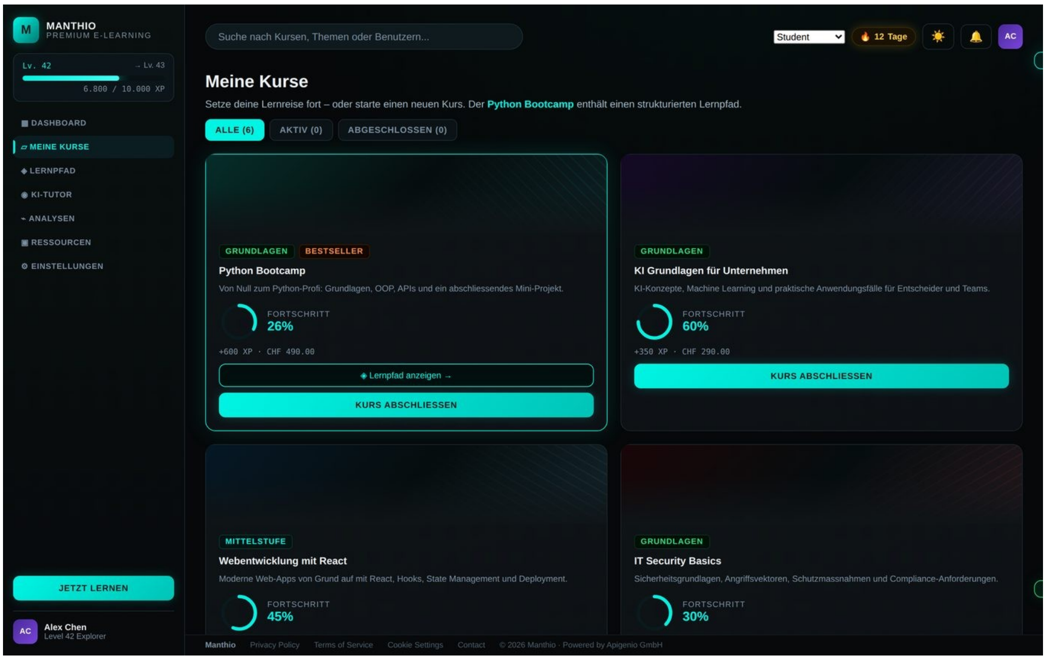
- Stats grid becomes a horizontal scroll of cards
- Two-column layout collapses to single column; right column items follow course preview
- Active course CTA remains prominent at top

11.6 Track Progress Widget (when learner is enrolled in a Career Track)

If the learner is enrolled in one or more Career Tracks (§14.7), the Dashboard surfaces a dedicated widget:

- Track name and outcome statement (compact)
- Overall track progress (e.g. "Course 2 of 4 · 38% complete")
- Next milestone label (e.g. "Up next: Prepare for KCNA")
- Primary CTA: "Continue Track" → jumps to next uncompleted lesson in the track
- Secondary action: "View full Track" → opens the Track visualisation page (§14.7)
- `REQ-DASH-020` If the learner is in multiple active Tracks, only the most-recently-active is shown by default; a toggle reveals others. [M]
- `REQ-DASH-021` If no Track is active but the learner has completed two or more courses, the Dashboard suggests a Track that includes those courses ("You've already done X and Y — these are part of the Z Track. Continue?"). [S]

12. Course Catalog (My Courses)



Student mockup reference — Course catalog grid with progress rings and level badges

12.1 Purpose

Help learners find what to learn next — either an individual course or a complete Career Track (§14.7). The catalog is the primary discovery surface and one of the main marketing pages of the platform.

12.2 Two Discovery Modes

The catalog has two tabs at the top:

- **Courses:** the traditional course list — browse, filter, search, enrol in single courses.
- **Tracks:** curated Career Tracks — multi-course sequences toward a defined outcome (see §14.7). For someone new to a topic area, Tracks are the recommended entry point.

On first visit, the default tab is Tracks — Tracks present a clearer narrative for new learners. Returning learners see the tab they used last.

The catalog serves two functions: showing the user their enrolled courses, and allowing them to discover new courses they could enrol in.

12.3 Layout Structure

- Page heading with sort/filter controls
- Tabs: "All courses" (entire catalog), "My courses" (enrolled), "Completed"
- Filters: Topic, Level (Beginner/Intermediate/Advanced), Format (Self-paced/Cohort), Duration, Language
- Grid of course cards (3 columns desktop, 2 tablet, 1 mobile)

12.4 Course Card

12.4.1 Content

- Course image / illustration / abstract gradient (per course)
- Course title
- Short description (2 lines, truncated)
- Level badge (Foundation, Intermediate, Advanced)
- Optional tag (Bestseller, New, Limited cohort)
- Format indicator (Self-paced / Cohort with start date)
- XP reward
- Price (CHF) or "Included in your plan" or "Provided by your employer"
- Progress ring (if enrolled, with percentage)
- Primary CTA varies by state: "Enrol", "Continue", "Review"

12.4.2 Requirements

- `REQ-CATALOG-001` Catalog default view is "My courses" if the user has any, otherwise "All courses". [M]
- `REQ-CATALOG-002` Filters update the result grid instantly without page reload. [M]
- `REQ-CATALOG-003` Sort options: Recommended, Newest, Most popular, Highest rated, Alphabetical. [M]
- `REQ-CATALOG-004` Active filter selections are visually displayed and clearable individually. [M]
- `REQ-CATALOG-005` Empty state when filters return no results includes a "Reset filters" action. [M]
- `REQ-CATALOG-006` Card click goes to Course Detail (§13). [M]
- `REQ-CATALOG-007` Primary CTA on the card is direct: "Continue" → Content Player; "Enrol" → Checkout; "Review" → Course Detail. [M]

12.5 Search Within Catalog

- `REQ-CATALOG-010` Search bar at top of catalog filters results in real time. [M]
- `REQ-CATALOG-011` Search matches course title, description, tags, and trainer name. [M]
- `REQ-CATALOG-012` Search supports typo tolerance and stemming. [S]

13. Course Detail and Booking

> *This area was not present in the student mockup. It is essential for both discovery and conversion.*

13.1 Purpose

The Course Detail page convinces a learner to enrol and provides the entry point for both purchase and continuation. It serves the discovery audience (not yet enrolled) and the enrolled learner (returning to review).

13.2 Sections (in order)

13.2.1 Hero

- Course title (large)
- Subtitle / value proposition (one sentence)
- Level badge, duration, language, format indicator
- Trainer thumbnail + name
- Rating (star average + count, if rated)
- Primary CTA: "Enrol now" (or "Continue learning" if enrolled)
- Visual: course illustration or trainer photo

13.2.2 What You'll Learn

- 4–8 specific learning outcomes as a checklist
- Each outcome is a single concrete skill or capability

13.2.3 Curriculum Preview

- Collapsible list of modules
- Each module shows: title, duration, brief description, lesson count
- First module can be previewed ("Preview first lesson" button)
- Lessons within a module are visible on expand

13.2.4 Format Details

Manthio supports three delivery modes. The Course Detail page adapts depending on which mode(s) are offered for a given course.

- **Self-paced view:** "Start anytime", estimated total duration, indicative time per week, list of included materials. No fixed dates, no in-person element.
- **Online cohort view:** Next available cohort start date, schedule of live video sessions, cohort size limit, current enrolment count, list of past cohort outcomes. Fully online.
- **Flipped (hybrid) view:** Combines online self-study units with in-person sessions at apigenio's training location or a partner venue. Shows: schedule of self-study windows (e.g. "Complete units 1–2 before session A"), dates and venue of in-person sessions, what to bring, prerequisites. This is the flagship format for digitised apigenio bootcamps.

- **Multi-mode view:** If a course offers more than one mode, the learner sees a comparison table at the top of the section and chooses at enrolment. Differences highlighted: price, AI Tutor allowance, peer cohort access, in-person component, certificate eligibility.

13.2.5 Trainer Profile

- Trainer photo, name, title
- Short bio (2–3 paragraphs)
- Other courses by this trainer
- Optional links: LinkedIn, personal site

13.2.6 Reviews and Testimonials

- Star rating breakdown (1–5 stars)
- Individual written reviews with reviewer name, date, course completion confirmation
- Verified completion badge on reviews
- Pagination or "Load more"
- Sort: Most recent, Highest rated, Most helpful

13.2.7 Frequently Asked Questions

- Collapsible accordion of common questions
- Trainer-curated

13.2.8 Pricing and Plans

Visible to non-enrolled users only:

- Course price (CHF)
- "Included in Premium" indicator if applicable
- Money-back guarantee (if offered)
- Group/team purchase option link

13.2.9 Related Courses

- 3–6 related courses as cards (same as catalog cards)

13.3 Booking and Checkout Flow

13.3.1 Self-Paced Course Checkout

- Click "Enrol now"
- If not signed in: redirect to sign in / sign up, then return
- Checkout screen: course summary, price, applicable taxes, payment method
- Payment processed (Stripe or similar)
- Confirmation screen with "Start learning" CTA, email receipt

13.3.2 Cohort Enrolment

- Click "Enrol now" on a specific cohort
- If not signed in: redirect to sign in / sign up
- Cohort intake form: relevant background questions (if trainer configures)

- Payment processing
- Confirmation: cohort schedule added to calendar, pre-cohort materials accessible, welcome email

13.3.3 Organisation-Provided Access

- User clicks "Enrol"
- System detects organisation entitlement
- Skip checkout; proceed directly to course
- Confirmation: "This course is provided by [Org name]"

13.3.4 Bundled Platform Access

Many courses include time-bounded access to the broader Manthio platform as part of their purchase. This is the recommended model for flipped apigenio bootcamps: the course price covers both the course itself and a follow-on subscription for continued AI Tutor use, resource access, and analytics. For example, the flipped Python Bootcamp includes 2 months of platform subscription with unlimited AI Tutor use after course completion.

- **`REQ-CHECKOUT-010`** Course offerings can bundle subscription access; the bundle terms (duration, included features) are displayed prominently before checkout. [M]
- **`REQ-CHECKOUT-011`** After course completion (or at enrolment, configurable per bundle), the bundled subscription auto-activates with a clear notification of its duration and what it includes. [M]
- **`REQ-CHECKOUT-012`** Bundled subscriptions appear in §21.1.3 (Subscription and Billing) with the source course, activation date, and end date clearly visible. [M]
- **`REQ-CHECKOUT-013`** Bundle pricing must clearly show the value of the included subscription separately from the course price ("CHF 490 includes 2 months Premium worth CHF 60"). [M]
- **`REQ-CHECKOUT-014`** When a bundled subscription is about to expire, the learner is offered a one-click renewal with the standard Premium price. [S]

13.3.5 Minimum Cohort Size and Conditional Confirmation

Flipped and cohort courses are not economically viable below a minimum participant count (typically 3, configurable per course). The booking flow must communicate this clearly to avoid surprises.

- **`REQ-CHECKOUT-015`** Cohort and flipped course offerings declare a minimum participant count; the Course Detail page shows current bookings and remaining slots toward minimum/maximum. [M]
- **`REQ-CHECKOUT-016`** Booking below the minimum places the learner in a "reserved" state with clear messaging: "Your seat is reserved. The course runs once 3 participants are booked. We'll confirm by [date]." [M]
- **`REQ-CHECKOUT-017`** If the minimum is not reached by a cutoff date (trainer-configurable, e.g. 7 days before start), the learner is notified of postponement or refund — never silently. [M]
- **`REQ-CHECKOUT-018`** Payment for reserved seats can be deferred until the course is confirmed (configurable), removing the risk for early bookers. [S]

13.3.6 Pre-Course and Pre-Session Preparation

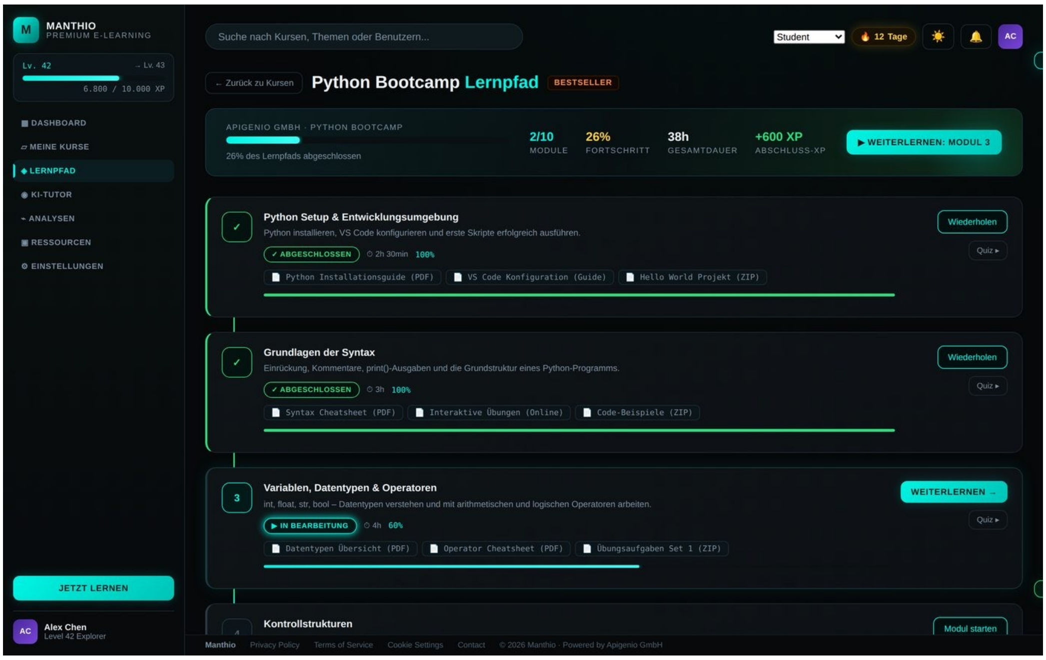
Flipped courses depend on learners arriving prepared. The Frontend must surface pre-course and pre-session prep clearly, not buried in the curriculum.

- **`REQ-CHECKOUT-020`** Course Detail page shows explicit pre-course requirements (laptop specs, software setup, prior knowledge) before checkout. [M]
- **`REQ-CHECKOUT-021`** After enrolment, a "Get ready" surface lists pre-course tasks (Setup verification quiz, intro video, prerequisites self-check). [M]
- **`REQ-CHECKOUT-022`** Each in-person session card in the Learning Path (§14) lists its prep modules — "Complete units 1–2 before this session" — as inline prerequisites with completion indicators. [M]
- **`REQ-CHECKOUT-023`** If a learner has not completed prep modules 24 hours before a live session, they receive a gentle reminder with one-click access to the prep content. [M]

13.3.7 Requirements

- **`REQ-CHECKOUT-001`** Checkout supports credit card, PayPal, and Swiss bank transfer (TWINT). [M]
- **`REQ-CHECKOUT-002`** Pricing displayed in user's local currency where possible; CHF is the default. [M]
- **`REQ-CHECKOUT-003`** Tax handling: Swiss VAT is added automatically for Swiss users; EU VAT MOSS handled. [M]
- **`REQ-CHECKOUT-004`** Promotional codes can be applied on the checkout screen. [M]
- **`REQ-CHECKOUT-005`** Cohort checkout shows cancellation policy clearly. [M]
- **`REQ-CHECKOUT-006`** Confirmation screen triggers gamification (welcome XP) and starts the course state. [M]
- **`REQ-CHECKOUT-007`** Failed payments show clear error messages with retry guidance. [M]

14. Learning Path



Student mockup reference — Learning path with module timeline and state-based card styling

---

The Learning Path is the structured spine of a course. It shows the sequence of modules, their states, and the learner's progress through them.

14.1 Purpose

Give the learner a clear, motivating map of where they've been, where they are, and what's next.

14.2 Layout Structure

- Header card with course title, overall progress bar, stats (modules completed, total hours, completion XP, etc.)
- Primary CTA: "Continue learning" → opens current module in Content Player
- Vertical timeline of modules with visual connectors
- Each module rendered as a card with state-specific styling

14.3 Module Card

14.3.1 Content

- Module number (or status icon: ✓ for completed, 🔒 for locked)
- Module title
- Description (2–3 lines)
- Status badge (Completed / In progress / Open / Locked)
- Type badge with distinctive icon (Self-study / Live online session / In-person session) — flipped courses mix all three
- Duration estimate (e.g. "1–2 hours" for self-study units, "4 hours" for in-person sessions)

- Scheduled time and venue (for live online or in-person modules; e.g. "Sat 18 May, 09:00–12:30 · apigenio Training Centre, Muri")
- Progress percentage (if started)
- Material chips (PDF, Video, Quiz, Code Sandbox indicators)
- Primary action button (state-dependent — "Start", "Continue", "Join session", "Get directions")
- Quiz access shortcut (if module has quiz)

14.3.2 States

State	Visual Treatment	Primary Action
Completed	Green check, faded card	Review (re-opens module)
In progress	Highlighted, progress bar visible	Continue
Open	Default styling, ready to start	Start module
Locked	Dimmed, lock icon	Disabled (with hint: complete prior module)

14.4 Functional Requirements

- `REQ-PATH-001` Modules unlock sequentially unless the course is configured to allow non-linear progression. [M]
- `REQ-PATH-002` Completing a module unlocks the next module immediately. [M]
- `REQ-PATH-003` Visual connectors between modules indicate completion flow (completed = filled, in-progress = animated, locked = dashed). [M]
- `REQ-PATH-004` Module materials can be previewed without entering the Content Player (hover or click to peek). [S]
- `REQ-PATH-005` Quiz shortcut on the module card opens the quiz directly without launching the Content Player. [M]
- `REQ-PATH-006` Completed module cards remain visible but with reduced visual weight. [M]
- `REQ-PATH-007` Locked modules show a tooltip explaining what's required to unlock. [M]

14.5 Cohort and Flipped Format Behaviour

- `REQ-PATH-010` In a fully online cohort course, modules unlock on a fixed schedule (e.g. Day 1 morning module unlocks Day 1 at 09:00). [M]
- `REQ-PATH-011` In a flipped course, self-study units unlock based on the planned schedule (typically before the related in-person session); in-person sessions appear on the path with their date, venue, and joining instructions. [M]
- `REQ-PATH-012` Future modules are visible but show "Available [date]" rather than "Locked". [M]
- `REQ-PATH-013` Live (online or in-person) session modules are clearly distinguished from self-study modules through type badge, icon, and dedicated scheduled-info area. [M]

- **REQ-PATH-014** For flipped courses, the path visualises the rhythm: "Self-study units 1–2 → In-person session A → Self-study units 3–4 → In-person session B". The visual flow should make the alternation obvious at a glance. [M]
- **REQ-PATH-015** In-person session cards include a "Get directions" link (opens map application) and "Add to calendar" action. [M]
- **REQ-PATH-016** Pre-session prep items ("complete unit 2 before this session") appear as inline prerequisites on the session card. [S]

14.6 Module Detail Expansion (Lesson-Level Building Blocks)

Module cards summarise; learners frequently want to see what is inside a module before committing. Clicking a module card expands it inline (or opens a detail panel on mobile) to reveal its constituent lessons — the building blocks of the course. This is where the learner discovers concretely what they will spend time on.

14.6.1 Module Detail Content

When a module card is expanded, the following is shown:

- **Module learning objectives:** 3–5 short statements ("By the end of this module, you can...") aligned to Bloom-level (§5.9). The objectives shape AI Tutor scope and quiz selection.
- **Lesson list:** ordered list of lessons inside the module, each with type icon, title, duration, status (completed / in progress / not started), and primary action.
- **Material summary:** aggregate counts ("3 videos · 2 articles · 5 exercises · 1 quiz") — gives the learner a quick mental model of the module.
- **Required vs optional indicator:** optional lessons (deep-dives, extra exercises) are visually distinguished from required ones. Optional lessons do not gate completion.
- **Estimated remaining time:** recalculated based on what the learner has already completed.

14.6.2 Lesson Card (the actual "Baustein")

Each lesson within a module is rendered as a compact card — the smallest navigable unit of the course:

- Type icon (Video, Article, Quiz, Code, H5P, Assignment, External, Live Event)
- Lesson title
- Duration (e.g. "8 min video", "15 min reading", "~20 min exercise")
- Status indicator (✓ completed, ▶️ in progress, ○ not started, 🔒 locked)
- Difficulty marker (1–3 dots, optional, trainer-set)
- Bloom-level chip (e.g. "Remember", "Apply") — subtle, for transparency
- Required/Optional badge if applicable
- Primary action: "Start" / "Continue" / "Review" (state-dependent)
- Secondary actions: bookmark, jump-to-Tutor with this lesson as context

14.6.3 Requirements

- **REQ-PATH-020** Module cards are expandable inline on desktop; on mobile they navigate to a Module Detail view. [M]
- **REQ-PATH-021** Lesson cards within an expanded module show all eight content fields listed above. [M]

- **`REQ-PATH-022`** Clicking a lesson card opens that lesson directly in the Content Player (§15). [M]
- **`REQ-PATH-023`** Optional lessons render with a clear "Optional" ribbon or chip; they appear in the path but do not count toward module completion. [M]
- **`REQ-PATH-024`** Locked lessons show a tooltip with the unlock condition (e.g. "Complete previous lesson" or "Available 18 May"). [M]
- **`REQ-PATH-025`** Lesson-level progress is preserved when the learner exits and re-enters mid-lesson. [M]
- **`REQ-PATH-026`** The expanded view should be skimmable in under 5 seconds — heavy density (long descriptions, dense badges) should be avoided. [M]

14.7 Career Tracks (Multi-Course Learning Paths)

> *This sub-section describes a feature that operates across multiple courses, not within one. It is its own page, accessed from the sidebar or the Course Catalog (§12), but is documented here because it is the natural extension of the single-course Learning Path concept.*

Once apigenio's catalog grows beyond a handful of courses (and especially when Manthio is offered to other trainers — see business model in §4), learners face a new problem: not "what's in this course?" but "which courses should I take, in what order, to reach my goal?"

Career Tracks answer this. A Track is a curated, opinionated sequence of courses that takes a learner from "I know nothing" to a defined outcome — typically a certification, a role-readiness level, or a project milestone. Reference implementations to study: KodeKloud's certification paths (CKA, CKAD), Coursera's Specializations, Pluralsight's Skill Paths.

14.7.1 Track Anatomy

- **Track title:** outcome-oriented, e.g. "Become a Certified Kubernetes Administrator", "From Zero to Python Production Engineer".
- **Track outcome statement:** one paragraph — what the learner can do at the end, what role/certification it prepares for.
- **Total estimated time:** aggregate of all included courses (e.g. "36 hours of learning + 2 in-person sessions").
- **Self-assessment entry:** at the top, the learner chooses their starting point ("I know nothing" / "I know the basics" / "I have some experience") — this determines which preparatory courses are highlighted vs. skipped.
- **Visual path:** a flowchart-style diagram showing the sequence of milestones, with course cards branching off each milestone.
- **Milestone nodes:** checkpoints on the track ("Start with basics", "Prepare for KCNA", "Get CKA Certified", "Capstone project") — these are not courses themselves but decision/progress points.
- **Course cards:** attached to each milestone, showing the course needed to reach it — thumbnail, title, duration, instructor, direct "Start" action, and "More details" link to Course Detail (§13).
- **Optional branches:** supplementary courses or mock exams marked as "Optional" with a distinctive ribbon.

14.7.2 Visual Treatment

The visual is as important as the function. A great Career Track page is one of the strongest acquisition surfaces a learning platform can have. References to internalise:

- Vertical or branched flow with clear directional connectors between milestones
- Distinct colour-coding per track or per skill family (subtle gradients, not loud)
- Course cards offset to the side of the main spine — left and right alternation reads well
- Current position of the learner highlighted with a subtle glow or animated marker
- Completed milestones visually distinguished (checked, faded but visible)
- Locked milestones grey and dashed-connector until prerequisites are met
- Each course card has a play button (direct start) and an external-link icon (Course Detail)

14.7.3 Self-Assessment at Entry

Inspired by the KodeKloud pattern: at the top of the track, the learner answers a short self-assessment that personalises the displayed path.

- **`REQ-TRACK-001`** Each Career Track has a self-assessment tab strip at the top: e.g. "I Know Nothing" / "I Know Basics" / "I Have Experience". Clicking a tab adjusts which courses are highlighted as needed vs. optional. [M]
- **`REQ-TRACK-002`** Self-assessment can also be performed via a diagnostic mini-quiz (5–8 questions) that recommends the appropriate starting tab — offered as an option, not forced. [S]
- **`REQ-TRACK-003`** The chosen entry level is persisted per learner per track; learners can change it at any time. [M]

14.7.4 Progress and Continuity

- **`REQ-TRACK-010`** When a learner enrols in a Track, their existing course completions count toward Track progress — they do not need to re-take courses they have already completed. [M]
- **`REQ-TRACK-011`** Track progress is displayed as a percentage at the top, with visual reinforcement on the path itself. [M]
- **`REQ-TRACK-012`** The next recommended course in the Track is also surfaced on the Dashboard (§11) as a "Continue your Track" card. [M]
- **`REQ-TRACK-013`** Completing a Track triggers a celebration ceremony comparable to completing a course (§9), plus a Track certificate (Phase 2 via Credly, §39). [M]

14.7.5 Track Catalog and Discovery

- **`REQ-TRACK-020`** Tracks are discoverable from the Course Catalog (§12) as a separate tab ("Tracks" alongside "Courses"). [M]
- **`REQ-TRACK-021`** Each Track has a public-facing landing page suitable for marketing — accessible without authentication. [M]
- **`REQ-TRACK-022`** Tracks can be filtered by goal type (Certification / Role / Project / Topic), estimated time, and difficulty. [M]
- **`REQ-TRACK-023`** Learners can bookmark Tracks before enrolling (signals interest, useful for marketing analytics). [S]

14.7.6 Authoring (out of scope here, signalled for completeness)

Track authoring lives in the Creator Studio (separate document), but the Learner Frontend constraints worth noting now:

- Tracks are composed of existing courses — they are not new content but new structure
- A course can appear in multiple Tracks
- Milestone nodes are author-defined waypoints; they can be aligned to external certifications (e.g. KCNA, CKA) or to internal capability labels
- Optional courses are marked at the Track level, not at the course level (a course can be optional in Track A and required in Track B)

14.7.7 Why Tracks Matter for apigenio's Strategy

Three distinct strategic uses:

- **Internal use (apigenio's own courses):** Once apigenio has 5+ courses (Python, JS, Data, AI, DevOps…), Tracks let you guide existing customers from one engagement to the next — "You finished Python; now follow the Data Engineer Track". Direct revenue multiplier.
- **B2B offering to organisations:** Companies licensing Manthio for their teams can compose their own internal Tracks ("Our Junior Developer Onboarding Track") that mix apigenio courses with their own custom content. Sticky enterprise feature.
- **Platform offering to external trainers:** Other trainers using Manthio can compose Tracks across their own catalog — and potentially cross-trainer Tracks if apigenio operates as a marketplace. This is the multi-tenant scaling angle (out of immediate scope, but Track architecture should accommodate it from day one).

> *Sections §14.7.8 through §14.7.11 below describe features that are newly conceived for Manthio. None of them appear in the student mockup. They are positioned here because they all extend the Career Track concept, and the design agency should treat them as opportunities for further differentiation rather than baseline requirements. All four are flagged as Phase 2 or Phase 3 — they do not gate the MVP and can be designed lightly in the first engagement, with detail added later.*

14.7.8 Peer Cohorts within Tracks (Phase 2, not in mockup)

Self-paced and Track-based learning suffers from one weakness that cohort-based learning solves naturally: isolation. A learner working alone through a Track loses the social fabric — the casual question, the shared frustration, the "oh you too?" moment that makes a learning journey stick. Peer Cohorts re-introduce this layer without forcing fixed schedules.

The mechanism is simple: when a learner enrols in a Track, the platform offers (opt-in) to join a peer cohort — a small group of 5–15 other learners who started the same Track within a similar time window. The group has its own scoped forum, can see each other's milestone-level progress (not lesson-level — that would feel surveillant), and can engage when they choose.

- **Joining is opt-in:** Learners who prefer solo work can decline. The default offer is shown after the first milestone is reached, not at enrolment (avoids the awkwardness of joining a group as a complete novice).
- **Visibility:** First name and last initial only by default; learners can opt into full name display. Anonymous mode also available ("Learner #4").
- **Communication surface:** A scoped forum thread per cohort, plus optional direct messages between cohort members. Uses the Community/Forum infrastructure (§20).

- **Progress visibility:** Each cohort member's current milestone is visible, presented as encouragement, not competition. No detailed lesson-level surveillance.
- **Cohort lifecycle:** A cohort exists for the expected duration of the Track plus 2 months. After that, it is archived but still accessible read-only — a reference of past discussions.
- **Batching:** Cohorts are formed automatically once 5 learners reach the first milestone within a rolling window (e.g. 14 days). Maximum 15 to keep conversations alive.
- `REQ-TRACK-030` Peer Cohort membership is opt-in and offered after the first Track milestone is completed. [M]
- `REQ-TRACK-031` Cohorts have a scoped forum thread visible only to members and the supervising trainer. [M]
- `REQ-TRACK-032` Cohort member progress is shown at milestone granularity only; lesson-level progress is private. [M]
- `REQ-TRACK-033` A learner can leave a cohort at any time; their past contributions remain visible but are attributed to "Former member". [M]
- `REQ-TRACK-034` Cohort size is capped at 15 to keep conversation density manageable. [M]
- `REQ-TRACK-035` After the expected Track duration, the cohort transitions to a read-only archive that remains discoverable to former members. [S]

14.7.9 Mentorship Pairing (Phase 3, not in mockup)

Once a Track has had several cohorts complete it, the platform has a valuable asset: learners who have just walked the same path, with the experience fresh in memory. They are uniquely positioned to mentor those starting out — and often want to, both for the recognition and because teaching consolidates their own understanding. Mentorship Pairing turns this latent value into an explicit feature.

A Track graduate can opt-in as a mentor. Learners in the early stages of the same Track can opt-in to be mentees. The platform pairs them based on shared timezone, language preference, and stated learning goal. Pairings agree to a light cadence — typically a 15–20 minute call every 1–2 weeks — and have a structured agenda template to make the time productive (rather than relying on small talk).

- **Mentor eligibility:** Completed the Track with a passing capstone or certification. Recent completion (within 12 months) preferred so the material is fresh.
- **Pairing algorithm:** Matches on language, timezone (within 4 hours), Track, and a goal-similarity check ("What do you most want to get out of this Track?").
- **Cadence:** Default is bi-weekly 15-minute call; pairs can choose other frequencies. Scheduled via the same infrastructure as Trainer Office Hours (§17.6).
- **Structured agenda:** Each session has an optional template: "What did you work on?", "Where did you get stuck?", "What's the goal for the next two weeks?". Not required, but offered by default.
- **Mentor recognition:** Mentors earn dedicated XP, a "Mentor" badge on their profile, and (optionally) extended platform access as recognition. This is what makes the model sustainable without paying mentors directly.
- **Quality safeguards:** Mentees rate sessions privately; mentors with consistent low ratings are removed from the pool. Trainers can review the program health.
- `REQ-TRACK-040` Mentor opt-in is offered after Track completion with a passing capstone or certification result. [M]
- `REQ-TRACK-041` Mentee opt-in is offered after the first Track milestone. [M]

- `REQ-TRACK-042` Pairing considers timezone (within 4 hours), language, and goal similarity. [M]
- `REQ-TRACK-043` Mentors earn dedicated XP and a Mentor badge on their profile. [M]
- `REQ-TRACK-044` Mentees can rate sessions privately; mentors with consistent low ratings are removed from the pool after a configurable threshold. [M]
- `REQ-TRACK-045` Mentors and mentees can dissolve a pairing at any time without penalty; rotation to a new pairing is offered. [M]

14.7.10 Capstone Projects (Phase 2, not in mockup)

A Track that ends with a final exam is less valuable than one that ends with something the learner can show. Capstone Projects are larger applied tasks — typically 4–10 hours of work — that synthesise everything learned in the Track into a single demonstrable piece of work. They serve three functions: they consolidate learning (Bloom: Apply / Create), they give the learner a portfolio piece, and they give apigenio a credible signal of competence beyond a quiz score.

For the apigenio Python Track, an example Capstone might be: "Build a command-line tool that ingests a CSV of expenses, categorises them via a configurable rules engine, and outputs a monthly summary. Submit the code, a README, and a 3-minute screencast walkthrough." The trainer defines the brief; the learner produces the artifact; the trainer (and optionally peers) review and provide feedback.

- **Project Brief:** Trainer-authored document with goals, deliverables, evaluation rubric, and time estimate. Brief is visible on the Track page once unlocked.
- **Submission surface:** Combines existing modules — Code Sandbox (§25) for code, File Upload (§27) for documents and screencasts, Notes (§28) for the README. Single "Submit for review" action when ready.
- **Review:** Trainer reviews each submission using a structured rubric (configurable per Capstone). Phase 3: peer review can supplement trainer review for scale.
- **Feedback:** Inline annotations on the code (Sandbox supports comments), plus an overall written assessment and a numerical rubric score.
- **Outcome:** Passing the Capstone unlocks the Track Certificate of Mastery (more weight than the standard course completion certificate). The Capstone artifact is added to the learner's profile as a portfolio piece, optionally public via a shareable URL.
- **Re-submission:** If the Capstone is not passed on first submission, structured feedback guides the learner on what to improve. Unlimited re-submissions allowed.
- `REQ-TRACK-050` Capstone projects appear as a distinct milestone at the end of a Track, visually marked with a star or trophy. [M]
- `REQ-TRACK-051` Capstone submissions can combine code, files, and screencasts in a single bundle. [M]
- `REQ-TRACK-052` Trainer review uses a configurable rubric; rubric scores are visible to the learner after review. [M]
- `REQ-TRACK-053` Inline code comments are supported in the Capstone review workflow. [M]
- `REQ-TRACK-054` Passing the Capstone unlocks the Track's Certificate of Mastery; failing allows unlimited re-submission with structured feedback. [M]
- `REQ-TRACK-055` Capstone artifacts can be optionally made public via a shareable URL for portfolio use. [S]

- **REQ-TRACK-056** Phase 3: peer review can supplement trainer review (1 trainer + 2 peer reviews per Capstone). [W]

14.7.11 Cross-Course Knowledge Graph (Phase 3, not in mockup)

As the catalog grows, learners face a different problem from "which Track?". They face: "I know X but not Y — where do I learn Y, ideally without re-doing X?". A Knowledge Graph addresses this by representing the platform's content not as courses, but as the concepts within courses, and the relationships between those concepts.

Every lesson is tagged with the concepts it teaches and the concepts it assumes. A concept is something like "loops", "list comprehensions", "REST APIs", "async/await", "Bayes' theorem". Concepts have relationships: "async/await" depends on "promises" which depends on "functions". When the learner asks "what should I learn next?", the platform can reason about gaps in their concept graph and recommend the most efficient way to fill them — sometimes that's a whole course, sometimes it's three specific lessons pulled from three different courses.

- **Tag taxonomy:** Trainers tag their lessons with concept labels from a shared (but extensible) taxonomy. Initial taxonomy seeded by apigenio for tech/AI/data topics; expandable by other trainers.
- **Concept relationships:** Trainers also declare prerequisite relationships between concepts ("loops" requires "variables"). The platform builds and maintains the DAG.
- **Learner concept graph:** As learners complete lessons, their personal concept graph fills in. Visualised as a network diagram on the Analytics page (§18), with mastered concepts highlighted.
- **Gap-based recommendations:** The AI Tutor's Just-in-Time recommendations (§31.7) use the concept graph rather than only course structure. "You need to know X for what you're doing — here's a 6-minute lesson on X from another course."
- **Concept search:** Learners can search the catalog by concept, not just by course title. "Show me everything on async/await" returns lessons from across the catalog ranked by relevance and learner's current level.
- **Cross-trainer interoperability:** If Manthio becomes a platform for multiple trainers, a shared concept taxonomy enables cross-trainer Tracks. Two trainers can co-create a Track that draws from each other's catalogs, with the graph ensuring concept continuity.
- **REQ-TRACK-060** Lessons can be tagged with concepts from a shared taxonomy; the Frontend exposes these tags subtly on the Lesson Card. [M]
- **REQ-TRACK-061** A learner's concept graph is visualised on the Analytics page (§18) as a network diagram or hierarchical view. [M]
- **REQ-TRACK-062** Concept search is available globally (Cmd/Ctrl+K, §29) and returns lessons across the entire accessible catalog. [M]
- **REQ-TRACK-063** AI Tutor recommendations leverage the concept graph for gap analysis. [M]
- **REQ-TRACK-064** The Knowledge Graph is read-only from the Learner Frontend; concept editing happens in Creator Studio (separate document). [M]
- **REQ-TRACK-065** Concept relationships display the prerequisite chain ("To learn async/await, you need promises, which need functions") with clickable navigation. [S]
- **REQ-TRACK-066** Privacy: a learner's concept graph is private by default; sharing publicly is opt-in. [M]

15. Content Player

> *The Content Player is the heart of the learner experience. It was not detailed in the student mockup but is the most-used screen in the platform. This section is intentionally long.*

15.1 Purpose

The Content Player delivers the actual learning content — videos, readings, code exercises, quizzes, interactive activities — while keeping the AI Tutor, navigation, and notes accessible without leaving the screen.

15.2 Layout Structure

Three-pane layout on desktop:

- **Left pane (collapsible curriculum):** Module list with current lesson highlighted, ~280px wide, collapsible to icon strip
- **Centre pane (content):** The actual learning content — adapts per content type
- **Right pane (AI Tutor + tools):** AI Tutor chat, notes, bookmarks, transcript — collapsible, ~360px wide

On tablet: left pane collapses by default; user expands as needed.

On mobile: full-screen content; left and right panes become bottom sheets accessed via toolbar.

15.3 Content Types Supported

The player must seamlessly render the following content types, each with its own controls and behaviour:

15.3.1 Video Lesson

- Embedded video player (see §23 for player spec)
- Closed captions toggle
- Playback speed control (0.5×, 0.75×, 1×, 1.25×, 1.5×, 2×)
- Chapter markers
- Transcript visible in right pane
- Picture-in-picture option

15.3.2 PDF Document

- Inline PDF viewer (see §24)
- Zoom, fit-to-width, full-screen controls
- Download option (if trainer permits)
- Text search within document
- Page navigation

15.3.3 Text / Article

- Rich-text rendering with Markdown source
- Code blocks with syntax highlighting

- Mathematical notation (LaTeX / KaTeX)
- Images with captions
- Estimated read time at top
- Reading progress indicator (scroll-based)

15.3.4 Inline Quiz

- Same UI as standalone quiz (see §22)
- Embedded within the lesson flow, not modal
- Multiple question types supported

15.3.5 Code Sandbox

- Browser-based code editor with execution (see §25)
- Pre-populated starter code
- Test cases visible
- "Run" and "Submit" actions

15.3.6 H5P Interactive

- Embedded H5P content via iframe or library integration (see §26)
- Common types: interactive video, branching scenario, drag-and-drop

15.3.7 Assignment / File Upload

- Assignment brief (text + optional file attachment)
- Submission area: file upload or text editor
- Submission deadline (if cohort)
- Feedback area (shown after submission)

15.3.8 External Link / Resource

- Curated external link with description and reason for inclusion
- "Open in new tab" — explicit, not surprise
- Marker for completion (user confirms they reviewed it)

15.4 Player Chrome (Universal Controls)

15.4.1 Top Bar of Player

- Course title (small, link back to Learning Path)
- Current module and lesson title
- Progress indicator (lesson X of Y, module progress bar)
- Toggle for curriculum pane
- Toggle for AI Tutor pane

15.4.2 Bottom Bar of Player

- "Previous lesson" button
- "Mark complete" or "Continue" button (depends on content type)
- "Next lesson" button (enabled only after current marked complete, configurable)

- Estimated time remaining in module

15.5 Curriculum Pane (Left)

- **`REQ-PLAYER-001`** The curriculum pane shows all modules and lessons in the current course. [M]
- **`REQ-PLAYER-002`** The current lesson is visually highlighted. [M]
- **`REQ-PLAYER-003`** Completed lessons show a check; locked lessons show a lock. [M]
- **`REQ-PLAYER-004`** Clicking a lesson navigates to it (if accessible). [M]
- **`REQ-PLAYER-005`** The pane is collapsible; the collapse state persists across sessions. [M]
- **`REQ-PLAYER-006`** Search within the curriculum is available. [S]

15.6 AI Tutor + Tools Pane (Right)

15.6.1 Tabs

- **AI Tutor:** Chat with context of the current lesson (see §16, §31)
- **Notes:** Personal notes for this lesson (see §28)
- **Transcript:** Searchable transcript (for video lessons)
- **Bookmarks:** Saved positions in the course
- **Resources:** Files attached to this lesson

15.6.2 Requirements

- **`REQ-PLAYER-010`** The right pane is collapsible and remembers state across sessions. [M]
- **`REQ-PLAYER-011`** Switching tabs in the right pane does not interrupt the centre content (video keeps playing, etc.). [M]
- **`REQ-PLAYER-012`** AI Tutor tab opens by default if the lesson type is complex (code, math); Notes tab default otherwise. [S]

15.7 Completion Mechanics

- **`REQ-PLAYER-020`** Each lesson has explicit completion criteria: watch X% of video, scroll to bottom of article, pass quiz, submit assignment, etc. [M]
- **`REQ-PLAYER-021`** When criteria are met, the "Mark complete" button becomes the primary action. [M]
- **`REQ-PLAYER-022`** On marking complete, lesson is checked off, XP awarded, and "Next lesson" becomes primary. [M]
- **`REQ-PLAYER-023`** If the lesson is the last in the module, completing it triggers module completion ceremony (confetti, badge, XP burst). [M]
- **`REQ-PLAYER-024`** If the lesson is the last in the course, completing it triggers course completion ceremony with certificate prompt (Phase 2 via Credly). [M]

15.8 Mobile-Specific Behaviour

- **`REQ-PLAYER-030`** On mobile, the centre content is full-screen by default. [M]
- **`REQ-PLAYER-031`** Curriculum pane is accessed via a list icon in the top-left, opens as a slide-in panel. [M]

- `REQ-PLAYER-032` AI Tutor is accessed via a chat bubble in the bottom-right, opens as a bottom sheet. [M]
- `REQ-PLAYER-033` Video player supports full-screen landscape orientation. [M]
- `REQ-PLAYER-034` Code sandbox is functional on mobile but with a notice recommending tablet/desktop for serious coding. [M]

15.9 Keyboard Shortcuts

- `REQ-PLAYER-040` Space: play/pause video. [M]
- `REQ-PLAYER-041` Left/Right arrows: skip 5 seconds in video; navigate quiz questions. [M]
- `REQ-PLAYER-042` C: toggle captions. [M]
- `REQ-PLAYER-043` F: full-screen toggle. [M]
- `REQ-PLAYER-044` Cmd/Ctrl+K: open AI Tutor with focus on input. [M]
- `REQ-PLAYER-045` ?: show shortcut help overlay. [S]

15.10 Microlearning Mode

Learner research (see §38) showed two distinct usage patterns: short sessions of 5–20 minutes (commute, breaks, lunch) and deep sessions of 1–3 hours (evening, weekend). The Player must serve both gracefully — a learner picking up the platform on a phone for 8 minutes should be able to make meaningful progress without feeling forced to commit to a full module.

- `REQ-PLAYER-050` Each lesson exposes its estimated time prominently before the learner starts; long lessons can be configured by the trainer as "micro-chunkable" (broken into 3–8 minute segments). [M]
- `REQ-PLAYER-051` A "Quick Session" mode on the Dashboard surfaces the next 5–15 minutes of available learning (a single video chunk, a short quiz, or a flashcard review), distinct from the "Continue learning" CTA which resumes the full module. [M]
- `REQ-PLAYER-052` Progress is saved at micro-checkpoint granularity, not just at module boundaries — closing the tab mid-segment never loses position. [M]
- `REQ-PLAYER-053` On mobile, the Player offers a "5 minutes free, what should I do?" surface in the AI Tutor that recommends the best micro-activity for the available time. [S]

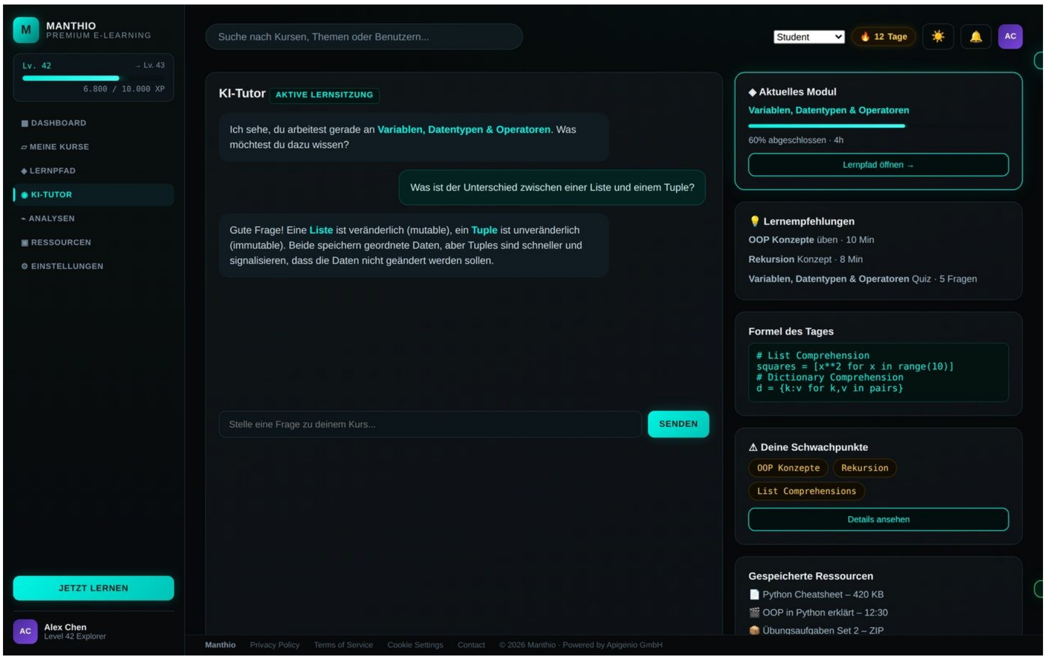
15.11 Auto-Completion and No Manual "Finish Course"

A clear finding from mockup testing: manual "Complete course" buttons confuse learners and feel unearned. Completion is a derived state, not an action.

- `REQ-PLAYER-060` Module completion is automatically triggered when all completion criteria of its lessons are met. No manual "mark complete" button at module level. [M]
- `REQ-PLAYER-061` Course completion is automatically triggered when all required modules are completed and any final assessment passed. No manual "finish course" button. [M]
- `REQ-PLAYER-062` Completion ceremonies (confetti, certificate prompt) play automatically the moment criteria are met, regardless of where the learner is on the platform. [M]
- `REQ-PLAYER-063` Lessons retain the "Mark complete" affordance only for content types where the system cannot detect completion automatically (e.g. reading an external link, watching offline). [M]

- **`REQ-PLAYER-064`** Once auto-completed, a course or module remains revisitable; the learner can re-watch, re-take quizzes, or re-do exercises without affecting completion status. [M]

16. AI Tutor



Student mockup reference — AI Tutor chat with contextual recommendations and code-of-the-day card

The AI Tutor is Manthio's signature differentiator. It exists both as a standalone page and as an embedded panel inside the Content Player.

16.1 Purpose

Provide patient, contextual, on-demand help. Replace the role of an in-person trainer for self-paced learners. Augment the trainer for cohort learners by handling routine questions.

16.2 Two Modes

- **Standalone view:** Full-page AI Tutor accessed from sidebar; broader conversation, can be used independently of a specific lesson
- **Embedded view:** Inside Content Player right pane; scoped to current lesson context by default

16.3 Standalone Page Layout

- Left: conversation history list (past chats, grouped by date)
- Centre: active chat with messages, input at bottom
- Right: contextual cards — current module, weaknesses, recommended exercises, saved snippets

16.4 Chat Interface

16.4.1 Message Types

- **User message:** Right-aligned, brand colour background

- **Tutor message:** Left-aligned, neutral background
- **Tutor reasoning indicator:** "Tutor is thinking..." while generating
- **System message:** Centred, italic — context changes or notices

16.4.2 Message Content

- Markdown rendering
- Code blocks with syntax highlighting and "Copy" button
- Mathematical notation (KaTeX)
- Inline references to course lessons (clickable, opens lesson)
- Image/diagram rendering (if Tutor generates)

16.4.3 Input

- Multi-line text input with auto-expand
- Submit on Enter; Shift+Enter for new line
- Suggested prompts above input (3 contextual quick-starts)
- Attachment button: paste code, upload screenshot (Phase 2)
- Voice input button (Phase 2: see §16.7)

16.5 Contextual Awareness

- **`REQ-TUTOR-001`** When opened from a lesson, the Tutor is automatically primed with that lesson's content as context. [M]
- **`REQ-TUTOR-002`** The Tutor's first message in a new lesson conversation acknowledges the lesson ("I see you're working on..."). [M]
- **`REQ-TUTOR-003`** The Tutor can reference and link to other lessons within the course. [M]
- **`REQ-TUTOR-004`** The Tutor knows the user's identified weaknesses and uses them to tailor explanations. [M]
- **`REQ-TUTOR-005`** The Tutor remembers conversation history within a session; cross-session memory is configurable in Settings. [M]

16.6 Tutor Capabilities (per stakeholder selection)

16.6.1 Core (Phase 1)

- Answer questions about lesson content
- Explain concepts with examples
- Provide hints on exercises without giving the answer
- Recommend related lessons or resources

16.6.2 Module-Specific Tutoring

- Knows the exact content of the current module
- Can quiz the user on the module material
- Identifies misconceptions and addresses them

16.6.3 Exercise Grading and Feedback

- Reviews submitted code or text

- Provides specific, line-by-line feedback
- Suggests improvements without full rewrites
- Awards partial credit for partial correctness

16.6.4 Adaptive Learning Path Suggestions

- Based on quiz results, recommends specific modules to revisit
- Based on time spent, suggests pacing changes
- Generates personalised review plans before exams or completion

16.7 Voice Mode (Phase 2)

Voice mode is a stated Phase 2 capability. The design system must accommodate it from day one.

- **`REQ-TUTOR-020`** A microphone button is visible in the input area; in Phase 1 it shows a "Coming soon" tooltip. [M]
- **`REQ-TUTOR-021`** Phase 2: Push-to-talk and continuous conversation modes. [W]
- **`REQ-TUTOR-022`** Phase 2: Voice output (Tutor speaks responses). [W]

16.8 Saving and Sharing

- **`REQ-TUTOR-030`** Any Tutor message can be saved as a note (sent to Notes pane of the relevant lesson). [M]
- **`REQ-TUTOR-031`** Code snippets in Tutor responses can be copied with one click. [M]
- **`REQ-TUTOR-032`** Conversations can be exported as Markdown. [S]
- **`REQ-TUTOR-033`** Useful exchanges can be marked as "helpful" — improves the Tutor over time. [M]

16.9 Limits and Transparency

- **`REQ-TUTOR-040`** If the Tutor doesn't know an answer, it says so clearly and offers next steps (ask in forum, contact trainer). [M]
- **`REQ-TUTOR-041`** The Tutor never invents facts about the course or trainers. [M]
- **`REQ-TUTOR-042`** A small disclaimer ("AI-generated, may make mistakes") is visible. [M]
- **`REQ-TUTOR-043`** Rate limiting is gentle and explained ("You've reached today's free Tutor limit — upgrade for unlimited"). [M]

16.9.1 Mitigating Cognitive Outsourcing

A consistent concern from trainers (validated in user research, see §38) is that learners may delegate thinking entirely to the AI Tutor — receiving answers without building understanding. The Tutor is designed to prevent this through specific behavioural rules, not by being less helpful.

- **`REQ-TUTOR-044`** On exercise-type questions ("how do I write this function", "why does my code fail"), the Tutor defaults to Socratic mode: it asks one targeted question or offers one hint, then waits — rather than supplying the full answer. [M]
- **`REQ-TUTOR-045`** The Tutor escalates gradually: hint → partial example → explanation → full solution, only progressing when the learner explicitly indicates they need more. [M]

- **`REQ-TUTOR-046`** After providing a substantial answer, the Tutor follows up with a brief comprehension probe ("Does this make sense — can you tell me in your own words why X happened?"). The learner can dismiss the probe but it is shown by default. [M]
- **`REQ-TUTOR-047`** During active assessment (quizzes, graded exercises), the Tutor refuses to provide direct answers and only offers conceptual guidance. The mode is clearly indicated to the learner. [M]
- **`REQ-TUTOR-048`** The Tutor never completes assignments end-to-end. It can review submitted work and suggest improvements, but it cannot be used as a ghostwriter. [M]

16.10 AI Source and Mode Indicator

Manthio's AI Tutor is backed by two engines (see §31.5): a self-hosted Local Document AI grounded in the current course materials, and a Cloud LLM for broader reasoning. The learner experiences a single chat interface, but the Frontend must surface which engine answered and let the learner influence routing.

16.10.1 Source Indicator on Tutor Messages

- **`REQ-TUTOR-050`** Every Tutor message displays a small badge indicating the source: "Course docs" (Local AI) or "Cloud AI" (general LLM). [M]
- **`REQ-TUTOR-051`** The badge is visually subtle but clearly readable; on hover/tap it shows a tooltip explaining what each source means. [M]
- **`REQ-TUTOR-052`** Local AI answers must list the source documents used (e.g. "Based on: Module 4, Section 2 · Lesson Transcript"); each listed source is clickable and opens the lesson at the relevant position. [M]
- **`REQ-TUTOR-053`** Cloud AI answers display a neutral source note ("General knowledge") without document links. [M]

16.10.2 Mode Toggle in Chat

Above or next to the chat input, a compact mode selector lets the learner influence which engine is used:

- **Auto (default):** System routes the question to whichever engine fits best (typically Local AI for course questions, Cloud for general/code/cross-domain)
- **Course docs only:** Forces Local AI. If the question is outside scope, the Tutor explains and offers to switch.
- **Full AI:** Forces Cloud LLM, useful for code generation, debugging, comparisons with outside knowledge.
- **`REQ-TUTOR-060`** Mode selector is visible at all times during chat. [M]
- **`REQ-TUTOR-061`** Mode preference is per-course and persists; switching courses resumes that course's last-used mode. [M]
- **`REQ-TUTOR-062`** The default mode for each course is set by the course author (trainers can mark a course as "Docs-heavy → default to Local" or "Open-ended → default to Auto"). [S]

16.10.3 Fallback Confirmation

When the Local AI determines a question is outside the scope of the indexed course documents, it doesn't silently call the Cloud LLM — it asks the learner first. This avoids surprise costs on metered plans and keeps the learner aware of which knowledge source is being used.

- `REQ-TUTOR-070` If Local AI cannot answer in Docs-only mode, the Tutor replies with a short message: "I can't find this in the course materials. Would you like me to ask the Cloud AI?" and offers a one-tap confirm. [M]
- `REQ-TUTOR-071` In Auto mode, low-confidence Local answers automatically escalate to Cloud, but the answer's source badge clearly shows "Cloud AI" with a note: "Not found in course docs; switched to general AI." [M]
- `REQ-TUTOR-072` Fallback never happens silently — the learner always sees that the source has changed. [M]

16.10.4 Streaming and Performance

- `REQ-TUTOR-080` Streaming UX must be identical whether the answer comes from Local AI or Cloud. The learner should not be aware of the underlying engine through latency alone. [M]
- `REQ-TUTOR-081` Local AI is expected to begin streaming within 500ms; Cloud within 2 seconds. The Frontend shows a generic "Tutor is thinking..." indicator in both cases. [M]

17. Live Session View

For cohort courses (online) with live video sessions, and for flipped courses with in-person workshop time, this view manages joining, participating in, and reviewing trainer-led events. It also covers asynchronous trainer access between sessions.

17.1 Three States

- **Pre-session:** Before start time
- **During session:** Active live event
- **Post-session:** After the event ends

17.2 Pre-Session View

- Session title and description
- Countdown to start
- Add to calendar (Google, Outlook, iCal)
- Pre-session materials ("Read these slides before joining")
- Trainer profile reminder
- "Join session" button (greyed out until 10 minutes before start)

17.3 Live Session View

- Embedded video conference (Zoom, Whereby, or similar)
- Side panel: chat, hand-raise, poll voting
- Materials accessible without leaving the session
- AI Tutor accessible for private questions during session

17.4 Post-Session View

- Recording (when available)
- Session notes / takeaways (auto-generated by AI from transcript)
- Polls results
- Follow-up assignments
- Trainer's office hours indicator

17.5 Requirements

- **`REQ-LIVE-001`** Video conferencing integration uses an established service (Zoom or Whereby); custom video infrastructure is out of scope. [M]
- **`REQ-LIVE-002`** Recording becomes available within 24 hours of session end. [M]
- **`REQ-LIVE-003`** Reminders are sent: 24h before, 1h before, 10min before. [M]
- **`REQ-LIVE-004`** Missed live sessions remain accessible as recordings. [M]
- **`REQ-LIVE-005`** AI-generated summary of the session is available as a study aid. [S]

17.6 Trainer Availability Between Sessions

In flipped and cohort formats, learners progress through self-study material between trainer-led sessions. During these windows, they need access to their trainer for questions, clarifications, and unblocking. This channel is distinct from the AI Tutor (which handles routine questions) and the Community Forum (peer help) — it is a direct, asynchronous line to the human expert leading the cohort. It is one of the explicit promises that justifies the price of a guided course over self-paced study.

17.6.1 Trainer Direct Messages

- In-app direct messaging to the cohort's trainer (one-to-one)
- Threaded conversations per topic so context stays attached
- Trainer's stated response window visible on the entry point ("Usually responds within 4 business hours")
- Trainer's current status (Available / Teaching / Away) visible to learners
- Code samples and screenshots attachable inline (uses the File Upload module, §27)

17.6.2 Trainer Office Hours

- Trainers publish recurring office-hour windows (e.g. "Tuesdays 17:00–18:00 CET")
- Learners book 10- or 15-minute slots via an inline calendar widget
- Office hours run on the same Live Session infrastructure as scheduled sessions
- Cancellation and rescheduling supported up to a configurable cutoff (e.g. 2h before)

17.6.3 AI Pre-Screening (optional, opt-in)

To respect trainer time and keep responses fast, learners can opt-in to have the AI Tutor pre-screen direct messages: if it can answer with high confidence from course documents, it drafts an answer and offers it to the learner before the message reaches the trainer. The learner stays in control — they can accept the AI draft, refine the question, or insist on sending to the trainer anyway.

17.6.4 Requirements

- **`REQ-LIVE-010`** Direct trainer messaging is available only to learners enrolled in the trainer's cohort or flipped course. [M]
- **`REQ-LIVE-011`** Trainer messages are not visible to other learners. [M]
- **`REQ-LIVE-012`** Trainers can set status (Available / Teaching / Away) with optional auto-reply ("Currently teaching, back at 14:00"). [M]
- **`REQ-LIVE-013`** Notifications to the trainer are throttled and digestable to avoid burnout (configurable per trainer). [S]
- **`REQ-LIVE-014`** AI pre-screening is opt-in per learner; defaults to off; can be toggled per message. [M]
- **`REQ-LIVE-015`** Trainers can mark a question as "Answer for cohort" — the response is also posted in the course forum so all learners benefit. [S]
- **`REQ-LIVE-016`** Office hour bookings appear in both the learner's and trainer's calendar (Add to Calendar / iCal feed). [M]
- **`REQ-LIVE-017`** Trainer Direct view is accessible from the Course Detail page, the Learning Path header, and the Live Session view. [M]

18. My Progress and Analytics

Student mockup reference — Analytics view with activity chart, skill profile, and weakness analysis

18.1 Purpose

Show learners their own learning data in motivating, useful ways. Help them identify what to work on next.

18.2 Sections

18.2.1 Stats Grid

- Total learning time (this week, this month, all time)
- Modules completed (across all courses)
- Current streak and longest streak
- Active subscription / spending

18.2.2 Activity Chart

- Daily learning minutes for the past 7 days (default) or 30 days (toggle)
- Comparative line: previous period overlaid
- Hover/tap reveals exact minutes per day

18.2.3 Weakness Analysis

- AI-identified gaps based on quiz and exercise performance
- Each weakness is clickable — opens AI Tutor in a focused session pre-loaded with: the topic, the learner's specific wrong-answer patterns, and a Socratic remediation plan (3–5 short prompts to close the gap)
- Severity indicator (light, moderate, significant gap)

- Recommended action per weakness: revisit lesson, do exercise set, ask Tutor, attend trainer office hours
- Spaced-repetition reinforcement: identified weaknesses automatically reappear as short review quizzes after 1, 3, 7 and 21 days until resolved

18.2.4 Skill Profile

- Bar chart or radar chart of competencies
- Scores derived from completed quizzes, exercises, and certifications
- Filtered by course or aggregated

18.2.5 Course Progress Overview

- Progress bar for each enrolled course
- Status: in progress, completed, paused
- Quick "Continue" actions

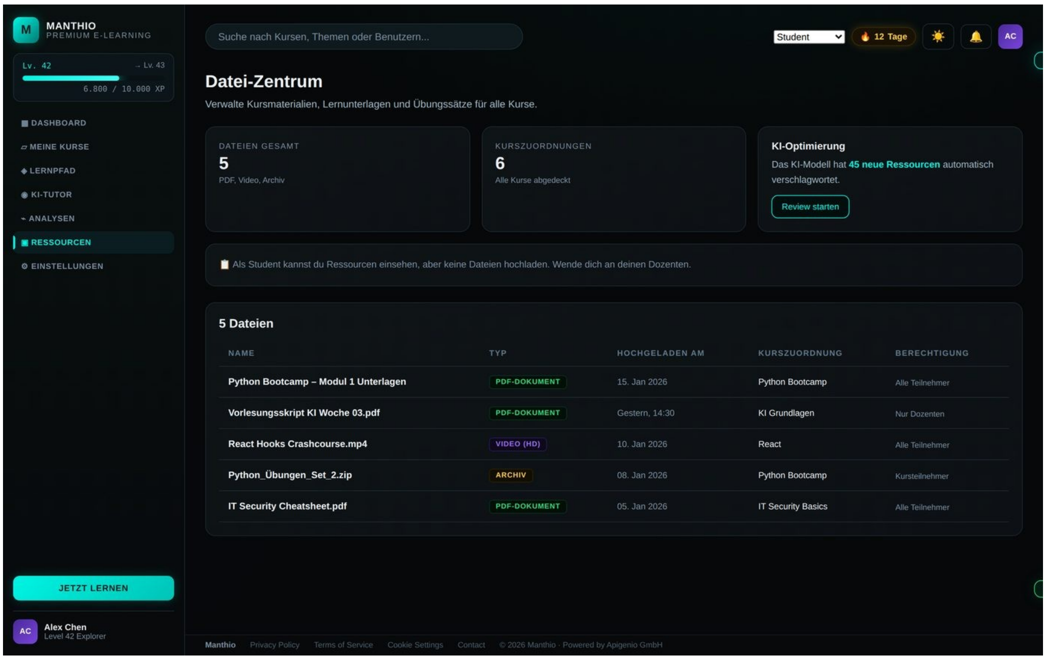
18.2.6 Learning Time Heatmap

- GitHub-style heatmap of learning activity over the past 90 days
- Days are colour-coded by intensity
- Hover/tap reveals total time and modules completed that day

18.3 Functional Requirements

- **`REQ-ANALYTICS-001`** All charts and data update in real time as the user completes activities. [M]
- **`REQ-ANALYTICS-002`** Data is exportable as CSV from a menu in the page header. [S]
- **`REQ-ANALYTICS-003`** Time-range selector (week / month / quarter / year / all time) at the top of the page. [M]
- **`REQ-ANALYTICS-004`** Comparison views (vs. previous period, vs. cohort average) are available. [S]
- **`REQ-ANALYTICS-005`** Privacy: cohort comparisons are anonymised and aggregate only. [M]

19. Resources and Knowledge Base



Student mockup reference — Resources file center with type badges and course association

19.1 Purpose

Centralised access to all files associated with the user's enrolled courses, plus a searchable knowledge base across the platform.

19.2 Two Views

- **My Files:** Files associated with courses the user is enrolled in
- **Knowledge Base:** Searchable cross-course content (Phase 2 feature)

19.3 My Files Layout

- Header: count of files, storage used (if applicable)
- Filters: file type, course, date range, access level
- Table or grid view of files
- Each file shows: name, type icon, course association, date, access level
- Actions: preview, download, save to notes, share (if allowed)

19.4 File Types Supported

- PDF documents (inline preview)
- Video (inline play)
- Archive (ZIP, download only)
- Notebook (Jupyter .ipynb — preview only)
- Code files (.py, .js, etc. — syntax-highlighted preview)
- Office documents (Word, PowerPoint, Excel — preview)

- Images (PNG, JPG)

19.5 Functional Requirements

- **REQ-RES-001** Files are organised by course by default; user can override sort. [M]
- **REQ-RES-002** Search within Resources covers filename and (where available) file content. [M]
- **REQ-RES-003** Files can be previewed without downloading where possible. [M]
- **REQ-RES-004** Download permissions respect trainer settings (some files preview-only). [M]
- **REQ-RES-005** Users can upload personal notes/files attached to a course (Phase 2). [S]
- **REQ-RES-006** Trainer-uploaded files appear automatically without learner action. [M]
- **REQ-RES-007** Removed from student mockup: "Upload" button for learners; learners do not upload course material, only assignments (see §27). [M]

20. Community and Forum

> *Community was a stated requirement but was not present in the student mockup. This section specifies a minimal viable community experience.*

20.1 Purpose

Allow learners to ask questions, help each other, and feel less alone — particularly in self-paced courses where the trainer is not immediately available.

20.2 Structure

- Forum per course (not platform-wide)
- Threads scoped to module or general
- Q&A model: question + accepted answer + supporting replies

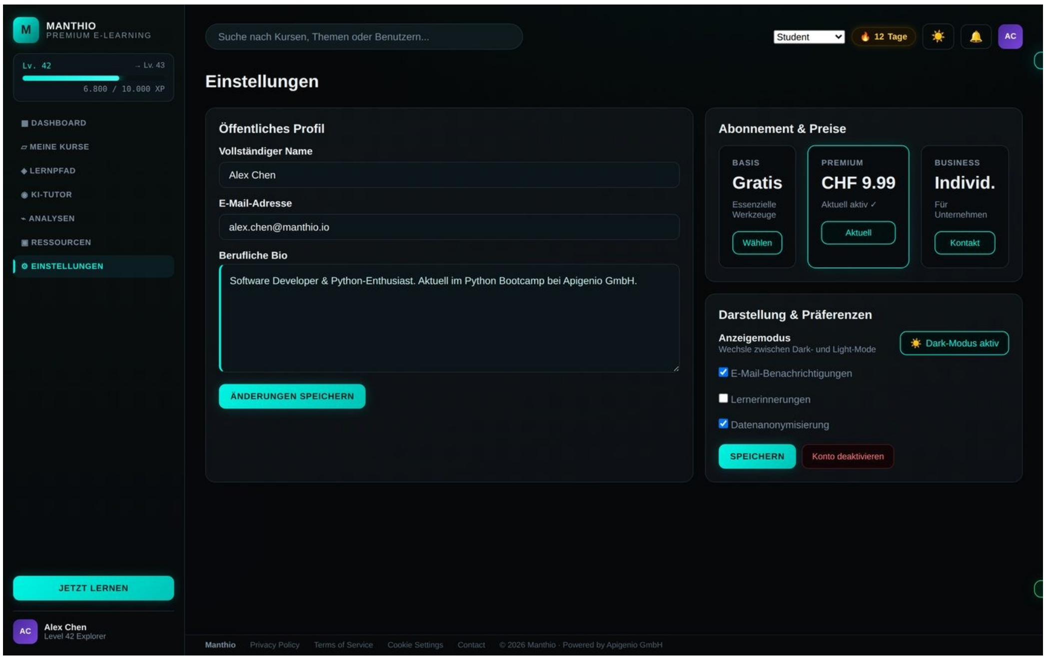
20.3 Thread

- Title (question)
- Body (Markdown, code blocks supported)
- Optional code attachment via Code Sandbox snippet
- Tags (module, topic)
- Voting (helpful / not helpful)
- Replies threaded one level
- Accepted answer indicator (set by original poster or moderator)
- AI Tutor auto-suggested answer (visible to all)

20.4 Functional Requirements

- **`REQ-FORUM-001`** Forum threads can be created from any module via a "Ask the community" button. [M]
- **`REQ-FORUM-002`** Posts can be anonymous within the forum (option per post). [M]
- **`REQ-FORUM-003`** AI Tutor automatically posts a suggested answer to every new thread within 30 seconds. [M]
- **`REQ-FORUM-004`** Trainers/cohort leads receive notifications for unanswered threads after 24 hours. [M]
- **`REQ-FORUM-005`** Marked-best answers award XP to the answerer. [M]
- **`REQ-FORUM-006`** Inappropriate content can be reported; reports trigger trainer/moderator review. [M]
- **`REQ-FORUM-007`** Subscription to threads triggers notifications on new replies. [M]

21. Profile and Settings



Student mockup reference — Settings with profile editor and subscription tier comparison

21.1 Sections

21.1.1 Public Profile

- Full name (display)
- Avatar (upload or initials)
- Professional bio (short)
- Visible courses (completed, in progress — user can hide)
- Profile visibility toggle (public / cohort only / private)

21.1.2 Account

- Email address (change requires re-verification)
- Password change
- Two-factor authentication setup
- Connected accounts (Google, Microsoft, etc.)
- Data export (download all your data as JSON)
- Delete account (with grace period)

21.1.3 Subscription and Billing

- Current plan (Basic / Premium / Business)
- Plan comparison and upgrade path
- Course-bundled subscriptions (e.g. "Python Bootcamp includes 2 months Premium") with source course, activation date, and end date
- Payment methods management
- Invoice/receipt history with download

- Cancel subscription

21.1.4 Preferences

- Theme (Dark / Light / System)
- Language (DE / EN)
- Email notifications (granular toggles)
- Push notifications (granular toggles)
- Reduced motion
- Sound effects on/off
- Default video playback speed
- AI Tutor default mode (Auto / Course docs only / Full Cloud AI) — with brief explanation of cost and privacy trade-offs (see §16.10, §31.5)

21.1.5 Privacy

- Data anonymisation for cohort comparisons
- AI Tutor conversation memory (on/off, per-session control)
- AI engine routing transparency: clearly stated that course-document questions stay on apigenio infrastructure (Local AI) and only out-of-scope questions reach Cloud LLM providers
- Marketing communications opt-in/out
- Analytics opt-in/out

21.2 Layout

- Two-pane layout: left = section navigation, right = section content
- On mobile: single column with collapsible sections

21.3 Functional Requirements

- **`REQ-SETTINGS-001`** Changes save automatically with a clear save indicator; explicit save buttons where appropriate (e.g. bio). [M]
- **`REQ-SETTINGS-002`** Destructive actions (delete account, cancel subscription) require typed confirmation. [M]
- **`REQ-SETTINGS-003`** Sensitive changes (email, password) require recent password re-entry. [M]
- **`REQ-SETTINGS-004`** Theme changes apply immediately without page reload. [M]
- **`REQ-SETTINGS-005`** Data export generates a downloadable archive within 24 hours and emails the user. [M]

Part 4 — Functional Modules

Functional modules are reusable components that appear across multiple pages. The design agency should treat each as a coherent component family — quiz styling and behaviour, for example, must be consistent whether the quiz appears inline in the Content Player, in the Learning Path, or as a standalone retrieval prompt.

- **§22 Quiz Engine:** question types plus retrieval practice and spaced repetition patterns.
- **§23 Video Player:** controls, captions, transcripts, mobile behaviour.
- **§24 PDF Viewer:** inline preview, search, notes.
- **§25 Code Sandbox:** Python/JS/SQL execution, worked-examples-and-faded-guidance pattern.
- **§26 H5P Player:** interactive content embedding.
- **§27 File Upload Module:** assignment submissions, avatar, attachments.
- **§28 Notes and Bookmarks:** personal layer over content.
- **§29 Global Search:** cross-content search via Cmd/Ctrl+K.
- **§30 Notifications:** in-app, email, push.
- **§31 AI Tutor Engine:** the hybrid Local + Cloud architecture with explicit routing and grounding behaviour.

22. Quiz Engine

22.1 Purpose

Deliver interactive assessment within lessons, between modules, and as final course exams. Supports multiple question types.

22.2 Question Types

- **Single choice (radio):** One correct answer from 2–6 options
- **Multiple choice (checkbox):** One or more correct answers
- **True/False**
- **Text input (short):** Pattern match or exact match
- **Code (Phase 2):** Submitted code is auto-evaluated against test cases
- **Drag-and-drop / matching (Phase 2)**
- **Hotspot / image labelling (Phase 3)**

22.3 Quiz Flow

- Quiz introduction screen: title, question count, estimated time, passing threshold, attempt count
- Question-by-question presentation
- Progress dots or bar at top
- Immediate feedback per question (configurable: show after each, or show all at end)
- Result screen: score, breakdown, explanation per question, retry option

22.4 Functional Requirements

- `REQ-QUIZ-001` Quiz can be configured to show feedback per question or only at end. [M]
- `REQ-QUIZ-002` Wrong answers display the correct answer and a hint/explanation from the trainer. [M]
- `REQ-QUIZ-003` Correct answers award XP per question (default 25 XP). [M]
- `REQ-QUIZ-004` Quizzes can be retaken; configurable cooldown and attempt limits. [M]
- `REQ-QUIZ-005` Time limits are optional per quiz. [M]
- `REQ-QUIZ-006` Answers can be saved as draft and resumed (for long quizzes). [M]
- `REQ-QUIZ-007` Quiz results are visible to the learner only; trainer sees aggregate. [M]
- `REQ-QUIZ-008` Passing thresholds gate module/course completion (configurable). [M]

22.5 Retrieval Practice and Spaced Repetition

Cognitive science evidence is strong: active recall outperforms passive re-reading, and spaced repetition stabilises memory far better than mass practice. Manthio uses both as built-in patterns, not optional add-ons.

22.5.1 Retrieval Prompts After Content

- `REQ-QUIZ-020` At the end of each content lesson (video, reading), a 1–3 question retrieval quiz appears automatically — quick, low-stakes, scored only as completion (not for grading). [M]
- `REQ-QUIZ-021` Retrieval quiz questions are generated by the Local AI from the lesson content (with trainer override). [S]
- `REQ-QUIZ-022` Skipping retrieval quizzes is allowed but tracked; the Analytics view shows correlation between skip rate and weakness emergence. [S]

22.5.2 Spaced Repetition Schedule

- `REQ-QUIZ-030` Items the learner answered incorrectly enter a spaced repetition queue with intervals: 1 day, 3 days, 7 days, 21 days. Correct answers advance the interval; incorrect resets it. [M]
- `REQ-QUIZ-031` A daily "Review" surface on the Dashboard shows what is due (typically 3–10 minutes of content). [M]
- `REQ-QUIZ-032` Spaced repetition items are pulled from across all enrolled courses, not just the active one. [M]
- `REQ-QUIZ-033` The schedule can be paused (vacation mode) without losing state. [S]
- `REQ-QUIZ-034` Identified weaknesses (§18) feed directly into the spaced repetition queue with higher priority. [M]

23. Video Player

23.1 Purpose

Deliver video lessons with controls and accessibility appropriate for learning.

23.2 Controls

- Play / Pause
- Skip backward 10s / forward 10s
- Playback speed (0.5×, 0.75×, 1×, 1.25×, 1.5×, 2×)
- Volume control with mute
- Full-screen toggle
- Picture-in-picture
- Closed captions toggle (with language selection)
- Quality selector (Auto, 1080p, 720p, 480p)
- Chapter markers (clickable)

23.3 Functional Requirements

- **`REQ-VIDEO-001`** Player remembers position; resuming a video returns to last-watched timestamp. [M]
- **`REQ-VIDEO-002`** Video progress is tracked; lesson is considered complete at 90% watched (configurable per lesson). [M]
- **`REQ-VIDEO-003`** Closed captions are required for every video; English and German minimum. [M]
- **`REQ-VIDEO-004`** Transcripts are available below or next to the player; clicking a line jumps to that timestamp. [M]
- **`REQ-VIDEO-005`** Adaptive bitrate streaming (HLS or DASH) ensures smooth playback. [M]
- **`REQ-VIDEO-006`** Mobile player supports full-screen landscape; gestures for skip and volume. [M]
- **`REQ-VIDEO-007`** Bookmarks can be created at the current timestamp from a button next to the player. [S]

24. PDF Viewer

24.1 Purpose

Display PDF lesson materials inline without forcing external downloads.

24.2 Controls

- Zoom in / out / fit to width / fit to page
- Page navigation (previous, next, jump to page)
- Thumbnail sidebar
- Text search within document
- Full-screen toggle
- Download (if permitted)
- Print (if permitted)

24.3 Functional Requirements

- `REQ-PDF-001` PDFs render within 2 seconds on the first page. [M]
- `REQ-PDF-002` Scroll position is preserved; resuming a PDF lesson returns to last-scrolled page. [M]
- `REQ-PDF-003` Lesson completion requires scrolling to the last page (configurable). [M]
- `REQ-PDF-004` Text within the PDF is selectable and copyable (unless trainer disables). [M]
- `REQ-PDF-005` Notes can be added at any page, anchored to the page. [S]

25. Code Sandbox

25.1 Purpose

Let learners write and execute code in the browser without setting up a local environment. Critical for the Tech / AI / Data Science target curriculum.

25.2 Supported Languages (Phase 1)

- Python (with NumPy, Pandas, Matplotlib pre-installed)
- JavaScript (Node + browser DOM)
- SQL (against sandboxed databases)

25.3 Phase 2 Languages

- R
- Bash / shell
- HTML/CSS

25.4 Components

- Code editor (Monaco or CodeMirror)
- Output panel (text and graphical)
- Test cases panel (visible to learner, with pass/fail indicators)
- File explorer (multi-file exercises)
- Run button
- Reset button (restore starter code)
- Submit button (sends to AI Tutor for grading)

25.5 Functional Requirements

- **`REQ-SANDBOX-001`** Code execution runs server-side; the learner does not have arbitrary code execution on the browser host. [M]
- **`REQ-SANDBOX-002`** Execution is rate-limited to prevent abuse. [M]
- **`REQ-SANDBOX-003`** Output appears within 3 seconds for simple code; long-running code is killed at 30 seconds with a clear message. [M]
- **`REQ-SANDBOX-004`** Code editor supports syntax highlighting, autocomplete, and linting. [M]
- **`REQ-SANDBOX-005`** Learner can save their solution; subsequent visits show their last saved version. [M]
- **`REQ-SANDBOX-006`** Test cases evaluation is automatic after Run; pass/fail visible. [M]
- **`REQ-SANDBOX-007`** Hidden test cases (not visible to learner) can be configured for evaluation. [M]
- **`REQ-SANDBOX-008`** Mobile experience: sandbox is functional but visually de-emphasises depth (encourages tablet/desktop). [M]

25.6 Worked Examples and Faded Guidance

Cognitive Load Theory shows that learners build skill fastest through a progression: fully worked example → partially worked example → independent attempt. The Sandbox supports this pattern as a first-class content authoring option.

- **`REQ-SANDBOX-010`** Trainers can publish an exercise as a three-stage sequence: (1) Worked example — fully explained solution the learner studies. (2) Faded — partially completed code with annotated blanks. (3) Independent — empty scaffold the learner completes. [M]
- **`REQ-SANDBOX-011`** Progression through stages is sequential by default; the learner unlocks the next stage by demonstrating engagement with the current one (e.g. predicting output, filling annotations). [M]
- **`REQ-SANDBOX-012`** AI Tutor recognises which stage the learner is on and adapts: at Worked stage it explains, at Faded stage it hints, at Independent stage it asks Socratic questions. [M]
- **`REQ-SANDBOX-013`** Learners can fall back to an earlier stage at any time if they get stuck; this is encouraged, not penalised. [M]

26. H5P Player

26.1 Purpose

Render interactive H5P content (videos with embedded questions, branching scenarios, drag-and-drops) as a first-class content type.

26.2 Content Types Required

- Interactive Video — video with overlay questions and seek prompts
- Branching Scenario — decision-tree learning
- Drag and Drop
- Fill in the Blanks
- Mark the Words
- Course Presentation (slide-based with embedded interactivity)

26.3 Functional Requirements

- **`REQ-H5P-001`** H5P content is embedded inline in the Content Player. [M]
- **`REQ-H5P-002`** Completion of H5P content is tracked and counted toward lesson progress. [M]
- **`REQ-H5P-003`** Scores from H5P contribute to XP and analytics. [M]
- **`REQ-H5P-004`** H5P content is mobile-responsive. [M]
- **`REQ-H5P-005`** Trainers can upload H5P packages; learners only consume. [M]

27. File Upload Module

27.1 Purpose

Allow learners to submit assignments and (in Phase 2) personal notes/files.

27.2 Upload Sources

- Assignment submission within a lesson
- Avatar upload
- Personal notes file (Phase 2)
- Forum thread attachment

27.3 Components

- Drop zone (drag-and-drop)
- File picker button
- Progress bar during upload
- Preview of uploaded file
- Remove / replace actions

27.4 Functional Requirements

- **`REQ-UPLOAD-001`** File size limit: 50 MB per file default; configurable per assignment. [M]
- **`REQ-UPLOAD-002`** File type restrictions are enforced per assignment configuration. [M]
- **`REQ-UPLOAD-003`** Virus scanning is performed on all uploads. [M]
- **`REQ-UPLOAD-004`** Failed uploads show clear errors with retry. [M]
- **`REQ-UPLOAD-005`** Multi-file uploads are supported with batch progress indicator. [M]
- **`REQ-UPLOAD-006`** Uploads can be resumed after interruption. [S]
- **`REQ-UPLOAD-007`** Confirmation toast on successful upload. [M]

28. Notes and Bookmarks

28.1 Notes Purpose

Personal scratchpad attached to lessons, modules, and courses. Learners' private thoughts, summaries, and references.

28.2 Notes Features

- Markdown editor with preview
- Note is anchored to a lesson or timestamp (in videos)
- Auto-save
- Searchable across all notes
- Tags
- Export to Markdown or PDF
- Save AI Tutor messages directly to notes

28.3 Bookmarks Purpose

Quick markers within content (video timestamps, PDF pages, code lines).

28.4 Bookmarks Features

- One-click bookmark button
- Bookmarks list per course
- Optional title and note
- Jump to bookmark from list

28.5 Functional Requirements

- **`REQ-NOTES-001`** Notes save automatically every 5 seconds. [M]
- **`REQ-NOTES-002`** Notes are private by default; sharing is opt-in. [M]
- **`REQ-NOTES-003`** Notes are exportable individually or in bulk. [M]
- **`REQ-NOTES-004`** Bookmarks sync across devices. [M]
- **`REQ-NOTES-005`** Notes support code blocks with syntax highlighting. [M]

29. Global Search

29.1 Purpose

Find anything on the platform from a single search field.

29.2 Searchable Entities

- Courses (title, description, tags)
- Modules and lessons
- Resources (files)
- Forum threads
- Notes (personal)
- AI Tutor conversations (personal)

29.3 Functional Requirements

- `REQ-SEARCH-001` Search input is in the top bar, accessible via Cmd/Ctrl+K. [M]
- `REQ-SEARCH-002` Results are grouped by entity type. [M]
- `REQ-SEARCH-003` Results appear in a dropdown/modal as the user types (debounced). [M]
- `REQ-SEARCH-004` Recently searched and recently visited items are shown when the search is empty. [M]
- `REQ-SEARCH-005` Keyboard navigation: up/down arrows, Enter to open. [M]
- `REQ-SEARCH-006` Typo tolerance and stemming are applied. [S]
- `REQ-SEARCH-007` Personal notes and Tutor history are only searched for the current user. [M]

30. Notifications

30.1 Channels

- In-app (bell icon dropdown)
- Email
- Push (web push for desktop, mobile app in Phase 3)

30.2 Notification Types

- **Course-related:** New module unlocked, assignment due, live session reminder, certificate ready
- **Social:** Reply to your forum thread, your answer marked best, mention
- **System:** Subscription renewing, password changed, login from new device
- **Gamification:** Streak in danger, milestone reached, badge earned
- **Marketing:** New courses available, special offers — opt-in

30.3 Functional Requirements

- `REQ-NOTIF-001` Notification preferences are configurable per type per channel. [M]
- `REQ-NOTIF-002` In-app notification dropdown shows the 20 most recent; older accessible on full notifications page. [M]
- `REQ-NOTIF-003` Notifications can be marked read individually or all at once. [M]
- `REQ-NOTIF-004` Critical notifications (live session about to start) override user's "do not disturb". [M]
- `REQ-NOTIF-005` Email notifications use a clean, branded template with clear unsubscribe link. [M]
- `REQ-NOTIF-006` Daily/weekly digest option for low-priority notifications. [S]

31. AI Tutor Engine

The technical engine behind the AI Tutor experience described in §16. While most of this is implementation detail, several aspects affect UX design.

31.1 Response Behaviour

- `REQ-AI-001` Tutor responses begin streaming within 2 seconds of submission. [M]
- `REQ-AI-002` Responses stream token-by-token for perceived speed; full response is rendered when complete. [M]
- `REQ-AI-003` Code blocks are properly formatted and copyable as they stream. [M]
- `REQ-AI-004` If the Tutor needs to call an external tool (e.g. execute code), this is shown to the user ("Running code..."). [M]

31.2 Context Management

- `REQ-AI-010` The Tutor has access to: current lesson content, course curriculum, user's quiz history (within this course), user's identified weaknesses, conversation history. [M]
- `REQ-AI-011` The Tutor does not have access to: other users' data, billing information, account credentials. [M]
- `REQ-AI-012` Long conversations are summarised internally to maintain context within token limits. [M]

31.3 Trust and Safety

- `REQ-AI-020` All Tutor responses are screened for prompt injection attempts. [M]
- `REQ-AI-021` User-provided content (code, text) is treated as data, not instructions. [M]
- `REQ-AI-022` Tutor refuses to provide answers that bypass assessment integrity (e.g. directly answering an active quiz question). [M]
- `REQ-AI-023` Tutor offers hints, partial solutions, and Socratic questioning rather than full answers when appropriate. [M]

31.4 Cost and Usage Management

- `REQ-AI-030` Tutor usage is metered and visible to the user ("X messages remaining today on the Basic plan"). [M]
- `REQ-AI-031` Free/Basic tier has a daily limit (e.g. 20 messages); Premium is unlimited. [M]
- `REQ-AI-032` When limits are reached, an upgrade prompt is shown gently. [M]

31.5 Hybrid AI Architecture (Local + Cloud)

Manthio's AI Tutor is built on a hybrid architecture combining a self-hosted Local LLM with a Cloud LLM provider. This is a deliberate choice driven by three factors: cost, privacy, and quality fit. The Local LLM answers most course-content questions cheaply and quickly; the Cloud LLM handles complex reasoning, code generation, and cross-domain questions where its larger context and capability are justified.

31.5.1 Rationale

- **Cost efficiency:** Course-content questions form the majority of Tutor traffic. Routing these to a small, self-hosted model reduces per-message API cost by an order of magnitude.
- **Privacy:** Course materials and conversation contents stay within Manthio's infrastructure for Local AI queries — no third-party LLM provider sees them.
- **Performance:** Local AI has lower latency for in-scope questions (no external API roundtrip).
- **Quality fit:** Document Q&A is a task where smaller models perform well when paired with good retrieval. Cloud LLMs are overkill for "What did module 4 say about decorators?".

31.5.2 Routing Logic (Backend, transparent to Frontend)

The Backend decides which engine answers each query. The Frontend doesn't make this decision but reflects it. Routing considers, in order:

- Explicit user mode (Docs only / Full AI / Auto) — see §16.10
- Question scope: is it answerable from indexed course documents?
- Confidence threshold: if Local AI's confidence is low, escalate to Cloud (with notification)
- Cost budget: free-tier users default to Local where possible
- Question type: code generation, debugging, cross-course comparisons → always Cloud

31.5.3 Frontend Responsibilities

Although routing is backend-driven, the Frontend has specific responsibilities to make the hybrid architecture transparent and controllable:

- `REQ-AI-040` Display source badge on every Tutor message (Local AI / Cloud AI). [M]
- `REQ-AI-041` Allow user to set mode preference per course (§16.10). [M]
- `REQ-AI-042` Surface fallback transitions explicitly — never silently switch sources. [M]
- `REQ-AI-043` Use identical streaming UX for both sources. [M]
- `REQ-AI-044` Make linked document references in Local AI answers clickable. [M]
- `REQ-AI-045` In Settings, expose a global default preference for AI mode (§21.1.4). [M]

31.6 Local Document AI (Course-Grounded RAG)

The Local Document AI is a small, self-hosted language model paired with a Retrieval-Augmented Generation (RAG) layer. It indexes the materials of the current course and answers questions grounded in those documents — quoting them where useful, with clickable source references.

31.6.1 What It Is

- **Model:** A small LLM (e.g. 3B–8B parameters) hosted on apigenio's infrastructure. Specific model is an engineering decision; the Frontend is model-agnostic.
- **Knowledge base:** Per-course corpus of all materials — lesson transcripts, PDFs, slides, code samples, trainer notes, captions. Excludes other learners' data and content from other courses (privacy).
- **Retrieval:** Embeddings stored in a vector database; questions are embedded and matched against the corpus; top-K relevant chunks are passed to the LLM as context.
- **Output:** Grounded answer with citations to source documents.

31.6.2 What It Does Well (default routing)

- Answering factual questions about course content ("What's the difference between a tuple and a list according to the lesson?")
- Summarising sections or modules ("Summarise the key takeaways of module 4")
- Locating content ("Where does the trainer explain decorators?")
- Quoting source material verbatim with location ("Quote the definition of List Comprehensions from the slides")
- Definition lookup ("What did the trainer mean by 'duck typing'?")

31.6.3 What It Doesn't Do (Cloud fallback)

- Code generation beyond what appears in course materials
- Debugging the learner's own code with general knowledge
- Cross-course or cross-domain comparisons ("How does this compare with React?")
- Open-ended career or strategic advice
- Translation, summarising external articles, anything not grounded in course docs

31.6.4 Document Indexing Lifecycle

- **`REQ-AI-060`** Course documents are indexed automatically when a learner first enrols in the course; subsequent indexing is incremental as the trainer adds materials. [M]
- **`REQ-AI-061`** Indexing covers: lesson text, PDFs, slide decks, video transcripts, code samples, downloadable resources. [M]
- **`REQ-AI-062`** Indexing excludes: assignments submitted by other learners, forum threads (privacy), trainer-private notes. [M]
- **`REQ-AI-063`** Indexing is per-user-per-course; embeddings include personalisation signals (notes, bookmarks) where the learner opts in. [S]
- **`REQ-AI-064`** When the trainer updates a course, the affected sections are re-indexed within 1 hour. [M]

31.6.5 Answer Quality and Grounding

- **`REQ-AI-070`** Every Local AI answer includes at least one document reference; answers without citations are flagged as low-confidence. [M]
- **`REQ-AI-071`** Document references include precise location: module, lesson, page or timestamp, paragraph anchor. [M]
- **`REQ-AI-072`** Verbatim quotes are clearly distinguished from paraphrase (typographic treatment: italics or quote block). [M]
- **`REQ-AI-073`** Local AI refuses to invent content not present in the documents; "I don't see this in the course materials" is a valid response. [M]
- **`REQ-AI-074`** Confidence scoring is exposed in metadata; the Frontend uses it for fallback decisions. [M]

31.6.6 Frontend Implications

- **`REQ-AI-080`** Document references in Local AI answers are inline clickable links; clicking opens the source lesson at the referenced position (page for PDFs, timestamp for videos). [M]

- `REQ-AI-081` A "Source documents" expandable section below each Local AI answer lists every document chunk that grounded the response. [M]
- `REQ-AI-082` Quoted material is visually distinct from generated commentary. [M]
- `REQ-AI-083` Local AI responses begin streaming within 500ms in typical conditions. [M]
- `REQ-AI-084` If the learner is offline, Local AI is unavailable (it runs server-side, not in the browser); the Frontend shows a clear offline state for the Tutor. [M]

31.6.7 Privacy Communication

Because the Local AI is a stated cost and privacy advantage, the Frontend should make this visible to learners in appropriate moments:

- `REQ-AI-090` The Settings privacy section explains that course-document questions are answered by a self-hosted model, not sent to external AI providers. [M]
- `REQ-AI-091` When a learner switches to "Course docs only" mode, a brief notice appears: "Your questions stay within Manthio." [S]
- `REQ-AI-092` An information icon next to the Local AI badge offers a one-line explanation suitable for non-technical users. [M]

31.7 Just-in-Time Learning Recommendations

A powerful but currently under-used affordance: the AI Tutor knows what the learner is doing right now (which lesson, which Track, which weakness pattern). It can proactively suggest the smallest piece of learning that helps the current moment — not the next lesson in sequence, but the lesson the learner needs right now.

31.7.1 Use Cases

- Learner is stuck on a code exercise → AI suggests a 4-minute lesson from another course that covers exactly the missing concept
- Learner asks a question outside the current course scope → AI suggests a relevant short clip from the catalog rather than answering from general knowledge
- Learner has 10 minutes free → AI surfaces the most impactful 8-minute lesson based on current weaknesses and Track progress
- Learner finishes a Track milestone → AI suggests the optimal next Track based on completed courses and stated goals

31.7.2 Requirements

- `REQ-AI-100` Tutor recommendations include explicit reasoning ("This 6-minute lesson on dictionaries directly addresses what you're stuck on"). [M]
- `REQ-AI-101` Recommendations are non-intrusive — they appear in the Tutor panel, not as modal popups. [M]
- `REQ-AI-102` Recommendations honour the learner's available time (when known via calendar integration or self-selected "5 minutes" / "30 minutes" buttons). [S]
- `REQ-AI-103` Learners can dismiss recommendations, and the system learns from dismissals (avoid same kind of suggestion). [S]
- `REQ-AI-104` Cross-course recommendations only point to courses the learner has access to (enrolled or free tier); locked courses are mentioned but with a clear "requires enrolment" badge. [M]

Part 5 — User Journeys

Static screens tell only half the story. These four journeys trace key flows through the platform, showing how the pages and modules connect over time. The agency should use them to validate that screen-by-screen designs hold together when stitched into real usage.

- **§32 New Learner:** Sara discovers Manthio, books her first course, lands in her first module.
- **§33 Active Learner:** Marc's 45-minute evening session — Dashboard → Continue → AI Tutor → Quiz → Done.
- **§34 Cohort Learner:** Tanya joins her first live session of the flipped Python Bootcamp.
- **§35 Stuck Learner:** Sara hits a wall; the AI Tutor proactively offers help and guides her without solving for her.

32. User Journey — New Learner: Discovery to First Module

32.1 Persona

Sara, the Upskilling Professional, never been on Manthio before.

32.2 Trigger

Sara sees a colleague post a Manthio course completion on LinkedIn. She clicks through.

32.3 Step-by-Step Flow

Step 1 — Landing on Course Detail (unauthenticated)

- She arrives at the Course Detail page (§13) for "Python Bootcamp"
- She reads the hero, scans "What You'll Learn", scrolls curriculum
- She checks the trainer profile and a few reviews
- She clicks "Enrol now"

Step 2 — Sign Up Prompt

- She is redirected to Sign Up (§10.1)
- She uses Google sign-on (one click)
- Email verification screen appears; she clicks the link in email

Step 3 — Onboarding

- Welcome screen briefly explains Manthio (§10.3.1)
- Goals questions captured (§10.3.2)
- She skips profile picture (§10.3.3)
- Recommendations show "Python Bootcamp" first (matching her intent)

Step 4 — Return to Checkout

- She clicks "Enrol" on Python Bootcamp
- Checkout (§13.3.1): course summary, CHF 490 + VAT
- She enters card details, completes payment
- Confirmation screen with confetti, "Start learning" CTA

Step 5 — First Module

- Clicking "Start learning" opens the Learning Path (§14)
- First module is unlocked, others are locked
- She clicks "Start module"
- Content Player (§15) opens with the first lesson (video)
- AI Tutor right pane greets her: "Welcome to Python Bootcamp! I'm here whenever you have questions."
- She watches the 8-minute video, reaches 90%, "Mark complete" becomes primary

- Clicks complete → +50 XP toast → next lesson is a 5-question quiz

32.4 Success Criteria

- Sara reaches her first lesson within 5 minutes of clicking "Enrol"
- She experiences zero confusion about what to do next at any point
- She has at least one positive emotional moment (confetti on enrolment, AI Tutor greeting)

32.5 Failure Paths

- **Payment fails:** Clear error message, retry option, alternative payment method suggested. Cart is preserved.
- **Email verification email delayed:** "Didn't receive it?" with resend button after 60 seconds. Alternative: contact support.
- **Onboarding skipped:** Dashboard has prominent "Complete your profile" and "Tell us your goals" prompts.

33. User Journey — Active Learner: Daily Session

33.1 Persona

Marc, the Self-Paced Explorer, currently 40% through "AI Fundamentals".

33.2 Trigger

It's 21:00. Marc has finished client work. He has 45 minutes for learning before bed.

33.3 Step-by-Step Flow

Step 1 — Open

- He opens manthio.com (already signed in)
- Dashboard (§11) greets: "Good evening, Marc 👋"
- Active course hero highlights: "AI Fundamentals — Continue with 'Embeddings explained'"
- Streak: 🔥 12 days. Stats grid visible."

Step 2 — Continue

- Clicks "Continue" on the hero
- Content Player opens directly to the lesson where he left off
- Video resumes at 4:23 (his last position)

Step 3 — Watch and Question

- He watches until 7:15, hits pause
- He has a question about cosine similarity vs dot product
- Clicks AI Tutor (right pane)
- Types: "Why use cosine similarity instead of plain dot product?"
- Tutor responds (knows current lesson context) with a clear explanation including formula
- Marc reads, saves the message to Notes

Step 4 — Complete and Practice

- Resumes video, finishes at 12:30
- Quiz appears (3 questions)
- Gets 2 of 3 right
- Tutor's explanation on the wrong one
- Marks lesson complete

Step 5 — Cooldown

- Toast: "+50 XP — Lesson complete!"
- Streak indicator updates to 🔥 13 days
- Player shows next lesson available; he clicks "Save for tomorrow" instead
- Returns to Dashboard

33.4 Success Criteria

- Marc completes a lesson and asks the AI Tutor at least one question
- Total session is under 30 minutes — efficient, no friction
- He maintains his streak

34. User Journey — Cohort Learner: Live Session

34.1 Persona

Tanya, the Bootcamp Participant, enrolled in a cohort-based Python Bootcamp starting today.

34.2 Trigger

It's 08:55. The first live session starts at 09:00.

34.3 Step-by-Step Flow

Step 1 — Pre-Session

- She received an email reminder at 08:00
- She opens Manthio at 08:55
- Dashboard shows: "🔴 Live session in 5 minutes — Python Bootcamp Day 1 Morning"
- Clicks the alert → Live Session View pre-session screen (§17.2)

Step 2 — Join

- Sees countdown: "Starting in 4:32"
- Reviews "Read before joining" slides (5 minutes earlier she'd been told to)
- At 08:58 "Join session" button becomes active
- Clicks, video conference opens in-frame (Whereby integration)

Step 3 — Participate

- Trainer's video and screen share fill the centre
- Chat side panel for cohort discussion
- AI Tutor available for private questions she doesn't want to ask in group chat
- She raises her hand once, trainer calls on her
- Poll appears mid-session, she votes

Step 4 — Post-Session

- Session ends at 12:30 (3.5 hours later)
- Live Session view transitions to post-session (§17.4)
- Notification: "Recording will be available within 24 hours"
- AI-generated summary appears within 30 minutes
- Afternoon's self-paced exercises are now unlocked in Learning Path

34.4 Success Criteria

- Tanya joins on time, never confused about where to go
- She participates (hand-raise, poll, chat)
- Post-session, she has clear next steps

35. User Journey — Stuck Learner: AI Recovery

35.1 Persona

Sara, mid-course, hits a wall on a code exercise.

35.2 Trigger

Sara has been on the same exercise for 25 minutes. Her code keeps failing the test cases.

35.3 Step-by-Step Flow

Step 1 — Recognise

- After 25 minutes on the same lesson, a subtle AI Tutor pulse appears
- Tutor proactively offers: "Looks like this exercise is challenging. Want a hint?"

Step 2 — Engage

- Sara clicks "Yes"
- Tutor analyses her current code (without showing the full answer)
- Says: "Your loop is iterating correctly, but you're not accumulating the result. Try adding a sum variable."

Step 3 — Iterate

- Sara modifies code, runs again — closer but still wrong
- Asks Tutor: "It's now returning 0"
- Tutor: "You're resetting the sum inside the loop. Move it outside."
- Sara fixes — passes all tests

Step 4 — Reinforce

- Tutor: "Nicely done. Quick question — why did moving sum outside fix it?"
- Sara answers in chat
- Tutor confirms her understanding, suggests two related exercises to reinforce
- XP: +75 (assignment) + 10 (Tutor reinforcement)

35.4 Success Criteria

- Sara doesn't give up — she completes the exercise
- She understands why it worked, not just that it worked
- She feels supported, not patronised

35.5 Anti-Patterns to Avoid

- Tutor giving the full answer too quickly — undermines learning
- Tutor being condescending ("This should be easy...")
- Tutor's proactive prompt being annoying / interrupting deep focus

Part 6 — Appendix

Reference material that supports the main document — definitions, scope boundaries, open questions for the design engagement, mockup mapping, and a worked example.

- **§36 Glossary:** key terms used throughout the document.
- **§37 Out-of-Scope:** what is excluded from this design engagement and why.
- **§38 Open Questions to the Design Agency:** decisions that remain open and require design input.
- **§39 Student Mockup Inventory:** transparency mapping of what was carried forward, added, or removed.
- **§40 Python Bootcamp Worked Example:** how the abstract specification translates into a concrete course offering.

36. Glossary

Term	Definition
Cohort	A group of learners who progress through a course together on a fixed schedule
Self-paced	A course consumption mode where the learner progresses at their own speed, without fixed dates
Flipped classroom	A blended learning model where foundations and guided exercises are studied online before the in-person session, and trainer time concentrates on application, debugging, and projects
Hybrid format	Used interchangeably with flipped classroom in this document; a course that mixes online self-study modules with in-person workshop sessions
Bundled subscription	A platform subscription included as part of a course purchase, typically time-bounded (e.g. 2 months Premium included with the Python Bootcamp)
Module	A grouping of lessons within a course; typically 30 minutes to 2 hours of content
Lesson	An atomic unit of content within a module (one video, one article, one quiz, etc.)
AI Tutor	The conversational AI assistant providing on-demand learning support
Local AI	The self-hosted small LLM that answers questions grounded in course documents (RAG-based)
Cloud LLM	External large-language-model service (e.g. Anthropic, OpenAI) used for broader reasoning and out-of-scope questions
RAG	Retrieval-Augmented Generation — a technique where a language model answers based on retrieved document chunks rather than its own internal knowledge
Embedding	A vector representation of text used to find semantically similar content; the basis of RAG retrieval
Trainer	The course author/instructor; may be apigenio or a third party
Cohort lead	A trainer or facilitator running a specific cohort instance
XP	Experience Points — gamification currency earned through learning activity
Streak	Consecutive days with at least one learning activity
H5P	An open-source content type framework for interactive learning content
Code Sandbox	Browser-based code editor with server-side execution
UC-03	Use Case 03 (referenced in student mockup) — Weakness identification feature
MVP	Minimum Viable Product — first releasable version
MoSCoW	Prioritisation method: Must, Should, Could, Won't

37. Out of Scope

37.1 Not in This Document

The following surfaces are explicitly excluded from this requirements specification. They will be specified separately:

- Creator Studio (the course-authoring interface for trainers)
- Organisation Admin (licence and user management for B2B customers)
- Platform Admin (back-office for apigenio operators)
- Marketing site (landing pages, course catalog for unauthenticated visitors)
- API documentation portal

37.2 Phase 2 Features (deferred)

Features in scope eventually but not for the initial design contract:

- Voice mode for AI Tutor (§16.7)
- Peer review between learners
- Real-world practice assignments with reflection workflow
- Native mobile applications (iOS, Android)
- Offline mode with content caching
- Credly integration for certificates
- Advanced analytics with cohort comparison
- Public leaderboards
- Multi-language for non-DE/EN audiences

37.3 Phase 3 Features (vision)

Features that may appear in long-term roadmap but are not specified here:

- AI-generated personalised micro-courses
- Cross-organisation learning analytics
- Certification governance (proctored exams)
- Marketplace for trainer-to-trainer content licensing

38. Open Questions to the Design Agency

The following questions remain open and should be addressed during the design engagement. Some are creative direction decisions; others are research questions that may require user testing.

38.1 Visual Identity

- Should the brand mark be the letter M, the Greek µ, or a more abstract symbol?
- Is the student mockup's cyan accent (#00f5e4) the right brand colour, or should we explore alternatives?
- Should we develop custom illustrations for course thumbnails or use abstract gradients?
- Should we have a mascot or persona character for the AI Tutor?

38.2 Interaction Patterns

- Sidebar collapsed by default on desktop or always expanded?
- AI Tutor: always visible on right pane in Content Player, or collapsed by default?
- Mobile primary navigation: hamburger only, or bottom tab bar as well?
- Confetti: how exuberant is too exuberant? Need to test with target audience.

38.3 Information Architecture

- Should "Resources" be a separate top-level navigation, or a tab within each course?
- Where does the user manage subscriptions — under Settings, or as a separate "Billing" page?
- Should the Learning Path be the default landing inside an enrolled course, or the first uncompleted lesson?

38.4 Edge Cases Needing Design

- Long course titles (50+ characters): how do they truncate, wrap, or scale?
- German vs English label length differences: max 30% expansion — does the layout hold?
- Empty states for new users on every page: what's the design pattern?
- Error states: 404, 500, network offline — what do they look like in our brand?

38.5 Content Strategy

- Should AI Tutor messages have suggested follow-up questions, or should the user always type?
- Tone of celebrations: "Well done!" vs "Crushed it!" — what fits Manthio?
- How honest should the platform be about AI Tutor limitations? "AI may be wrong" disclaimers — where and how often?

39. Reference: Student Mockup Inventory

This appendix maps requirements in this document back to the original student mockup (index.html, app.js, style.css). It is intended as a transparency reference, not a contractual artifact.

39.1 Features Carried Forward from Mockup

- Sidebar with primary navigation and XP/level indicator
- Dashboard layout with hero, stats grid, and two-column body
- Course Catalog with course cards
- Learning Path with module timeline and four module states
- Module quizzes and course-level quizzes with multiple choice + feedback
- AI Tutor chat interface (basic version)
- Analytics with weekly activity chart, weakness analysis, skill profile, heatmap
- Resources file table
- Profile and Settings with subscription tiers
- Modal system (level up, course completion, weakness detail, student detail)
- Toast notifications for XP and uploads
- Confetti animation on module/course completion
- Streak indicator in top bar
- Theme toggle (dark/light)
- Mobile hamburger menu and overlay

39.2 Features Added Beyond Mockup

Newly conceived during the requirements engagement — not present in the student mockup. The agency should treat these as additions to design from scratch.

39.2.1 Pages and Surfaces

- Authentication and onboarding flow (§10)
- Course Detail and Booking page (§13)
- Content Player as a dedicated, full-featured surface (§15)
- Live Session view for cohorts and trainer availability (§17)
- Community / Forum (§20)

39.2.2 Functional Modules

- Code Sandbox with worked-examples-and-faded-guidance progression (§25)
- H5P Player (§26)
- File Upload module (§27)
- Notes and Bookmarks (§28)
- Global Search beyond basic title filtering (§29)
- Comprehensive Notifications system (§30)
- Hybrid AI Tutor architecture: Local RAG + Cloud LLM (§31.5, §31.6)

39.2.3 Strategic / Pedagogical Layers

- Bloom's Taxonomy as the explicit didactic backbone (§5.9)
- Multi-modal courses: self-paced, online cohort, flipped/hybrid (§13.2.4)
- Bundled subscriptions tied to course purchases (§13.3.4)
- Minimum cohort size and reservation logic (§13.3.5)
- Pre-session preparation materials (§13.3.6)
- AI Tutor context awareness, grading, and cognitive outsourcing safeguards (§16)
- Microlearning Mode and Quick Sessions (§15.10)
- Auto-completion (no manual "Finish course") (§15.11)
- Retrieval Practice and Spaced Repetition (§22.5)

39.2.4 Multi-Course and Social Layers (mostly Phase 2 / 3)

The following features were proposed specifically to scale the platform from single-course usage to a multi-course, multi-trainer, multi-organisation product. None are in the student mockup; they require fresh design.

- Module Detail Expansion with lesson-level cards / Bausteine (§14.6)
- Career Tracks — multi-course curated learning paths (§14.7)
- Track Self-Assessment at entry (§14.7.3)
- Peer Cohorts within Tracks (§14.7.8 — Phase 2)
- Mentorship Pairing for Track graduates and newcomers (§14.7.9 — Phase 3)
- Capstone Projects as Track endpoints (§14.7.10 — Phase 2)
- Cross-Course Knowledge Graph and concept-based recommendations (§14.7.11 — Phase 3)
- Just-in-Time Learning Recommendations from the AI Tutor (§31.7)
- Track Progress Widget on the Dashboard (§11.6)

39.3 Features Removed from Mockup

- Role switcher in top bar (Student/Dozent/Admin) — roles are now determined by permissions, not user choice
- Dozent (trainer) and Admin views — moved to separate documents
- Student detail modal viewable by learners — relevant only in trainer views
- "Upload file" button for learners in Resources — replaced by assignment-submission upload (§27)

39.4 Notes on Visual Direction

The student mockup uses a strong cyber/tech aesthetic with cyan on near-black. This direction is acceptable but should be refined:

- Body font size needs to increase from 13px to 14–15px
- Light mode treatment is functional but lacks the same character as dark mode — needs design love
- Iconography in the student mockup uses Unicode characters (▦ ▱ ◈) — should be replaced with proper icon system
- Course images are placeholder gradients — final design needs a system for illustrations

39.5 Acknowledgement

The student mockup is a significant piece of work that demonstrates strong functional thinking. This requirements document builds on its foundation rather than discarding it.

40. Reference: Python Bootcamp as a Flipped Implementation

This appendix shows how the abstract specification of the platform translates into a concrete course offering. It is intended as a worked example for the design agency — when designing the Course Detail, Learning Path, and Module Card surfaces, this is the kind of course they should imagine.

40.1 Course Format

- **Mode:** Flipped (hybrid)
- **Self-study load:** 7 modules at approximately 1.5 hours each (≈10.5 hours total)
- **In-person time:** 2 half-day workshop sessions at apigenio's training venue
- **Bundled subscription:** 2 months of Manthio Premium access (AI Tutor unlimited, full analytics) after course completion
- **Minimum cohort:** 3 participants; comfortable size 4–8
- **Price:** CHF 1'000 per participant standard; group discounts from 6 participants

40.2 Module Structure

Module	Format	Duration	Bloom level	Cognitive justification
1. Setup & Python Basics	Self-study	1.5 h	Remember/ Understand	Standardised setup is ideal for guided self-study
2. Operators, Strings & I/O	Self-study	1.5 h	Remember/ Understand	Short guided exercises with instant feedback
Workshop A (modules 1–2 review)	In-person	0.5 day	Apply/Analyse	Setup verification, first script live, debugging together
3. Data Structures	Self-study	1.5 h	Understand/ Apply	Operations practised at own pace; analytics flag struggles
4. Control Flow & Comprehensions	Self-study	1.5 h	Understand/ Apply	Repetition-heavy; platform supports with quiz cycles
5. Functions & Scope	Self-study	1.5 h	Understand/ Apply	Building blocks for module 8 — must be solid
6. Modules, Packages & Environments	Self-study	1.5 h	Understand	Step-by-step instructions, tooling-heavy
7. Error Handling & File I/O	Self-study	1.5 h	Understand/ Apply	Concrete examples and patterns work

				well online
Workshop B (Mini-project)	In-person	0.5 day	Apply/ Evaluate/ Create	CLI application, pair programming, code review, feedback

40.3 Why This Maps to the Spec

- **§13.2.4 Flipped view:** This is the canonical flipped course. Course Detail shows the 7-module self-study schedule plus 2 in-person dates.
- **§13.3.4 Bundled Platform Access:** CHF 1'000 includes 2 months Premium worth approximately CHF 60.
- **§13.3.5 Minimum Cohort Size:** 3-participant minimum visible during booking.
- **§13.3.6 Pre-Session Prep:** Workshop A's card lists modules 1–2 as inline prerequisites.
- **§14 Learning Path:** Visualises the rhythm — 2 self-study modules → Workshop A → 5 self-study modules → Workshop B.
- **§17.6 Trainer Availability:** Trainer available asynchronously during self-study weeks; office hours posted for week 1 and week 2.
- **§25.6 Worked Examples:** Modules 5 (Functions) and 8 (Mini-project) use the worked → faded → independent progression.

40.4 What Success Looks Like

- All 7 self-study modules completed before Workshop B
- Quiz pass rate ≥ 80% per module
- Workshop A attendance with prep work done by ≥ 90% of participants
- Mini-project completed and reviewed live in Workshop B
- ≥ 50% of participants continue using Manthio Premium beyond the 2-month bundle

— End of Document —